

Rapport de Projet Fin d'Année

**Digitalisation des services de restauration :
Système de réservation, gestion des commandes et
moteur de recommandation intelligent**

**Année Universitaire :
2025 – 2026**

Réalisé par :

DIOURI Mehdi

EL KHARRAZI Ibtihal

Encadré par :

Mme. BENSALEH Nouhaila

Dédicaces

Nous dédions ce travail à nos familles, qui nous ont soutenus tout au long de ce projet. Leur patience, leur confiance et leurs encouragements nous ont aidés à avancer, surtout pendant les périodes les plus chargées.

Nous le dédions aussi à nos enseignants, pour les connaissances qu'ils nous ont transmises en génie logiciel, développement web, bases de données, modélisation, sécurité et gestion de projet.

Enfin, nous pensons à toutes les personnes qui nous ont encouragés à apprendre par la pratique, à corriger nos erreurs et à progresser étape par étape.

Remerciements

Avant de présenter le projet Tastify, nous tenons à remercier nos familles pour leur soutien tout au long de notre formation. Leur présence et leurs encouragements nous ont aidés à avancer avec sérieux jusqu'à la réalisation de ce Projet de Fin d'Année.

Nous remercions aussi l'ensemble du personnel enseignant et administratif pour le cadre de travail qu'il nous a offert durant cette année.

Nous adressons un remerciement particulier aux enseignants qui nous ont accompagnés dans les matières liées à ce projet : conception UML, développement backend, développement frontend, architecture logicielle, bases de données, tests et documentation technique. Les connaissances acquises dans ces modules nous ont permis de concevoir et de réaliser Tastify dans de bonnes conditions.

Résumé

Tastify est une application web full-stack destinée à centraliser la gestion quotidienne d'un restaurant. Le projet répond à des difficultés concrètes observées dans la restauration : dispersion des informations, erreurs de transmission entre la salle et la cuisine, suivi manuel des stocks, manque de visibilité sur les réservations, encaissements complexes et exploitation limitée des avis clients.

La solution repose sur un backend Django REST Framework, une base de données MySQL, Redis pour les échanges temps réel et les traitements asynchrones, Celery pour les tâches de fond, ainsi que deux applications React : une interface staff pour le gérant, le serveur et le cuisinier, et un portail client. Les modules réalisés couvrent le menu, les tables, les commandes, le Kitchen Display System, les stocks, les ressources humaines, les réservations, les paiements, la fidélité, les avis, les tableaux de bord et la configuration du restaurant.

Une partie importante du projet concerne l'analyse de sentiment des avis clients. Le code implémente un pipeline asynchrone qui analyse les commentaires au moyen de l'API Hugging Face lorsque le jeton est configuré. Il utilise le modèle multilingue NLP Town pour les avis non arabes, MARBERT pour les commentaires en arabe, et un secours local par mots-clés nommé `mots-cles-simple`. Les identifiants exacts des modèles sont détaillés dans le chapitre consacré à l'analyse de sentiment. Cette fonctionnalité aide le gérant à suivre la satisfaction client sans interrompre l'expérience utilisateur.

Mots-clés : ERP, restaurant, Django, React, MySQL, Redis, Celery, WebSocket, KDS, avis clients, analyse de sentiment, Hugging Face, PFA.

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface.
ASGI	Asynchronous Server Gateway Interface.
CRUD	Create, Read, Update, Delete.
DRF	Django REST Framework.
ERP	Enterprise Resource Planning, progiciel de gestion intégré.
JWT	JSON Web Token.
KDS	Kitchen Display System, écran de suivi des commandes en cuisine.
ML	Machine Learning, apprentissage automatique.
NLP	Natural Language Processing, traitement automatique du langage naturel.
ORM	Object-Relational Mapping.
PFA	Projet de Fin d'Année.
PWA	Progressive Web App.
RBAC	Role-Based Access Control, contrôle d'accès basé sur les rôles.
REST	Representational State Transfer.
SPA	Single Page Application.
SQL	Structured Query Language.
UML	Unified Modeling Language.
WS	WebSocket.

Table des figures

2.1	Diagramme de cas d'utilisation du back-office gérant	10
2.2	Diagramme de cas d'utilisation des modules salle et cuisine	11
2.3	Diagramme de cas d'utilisation du portail client	12
2.4	Diagramme de classes de Tastify	13
2.5	Diagramme de séquence de l'authentification staff	14
2.6	Diagramme de séquence de la prise de commande et envoi au KDS	15
2.7	Diagramme de séquence de la création d'un avis et analyse de sentiment	15
2.8	Architecture fonctionnelle générale de Tastify	17
3.1	Pipeline technique de l'analyse de sentiment	23
3.2	Back-office : avis clients avec scores de sentiment	24
4.1	Architecture technique de Tastify	32
4.2	Séquence simplifiée d'authentification JWT	34
4.3	Flux temps réel entre prise de commande et KDS	34
4.4	Connexion staff : accès sécurisé au back-office	37
4.5	Tableau de bord gérant du back-office	37
4.6	Interface serveur : plan de salle	38
4.7	Kitchen Display System côté cuisine	38
4.8	Portail client : consultation du menu	39
4.9	Portail client : plats recommandés classés selon les avis positifs	40
4.10	Back-office : gestion du stock et alertes d'ingrédients	41
4.11	Portail client : réservation d'une table	41
4.12	Portail client : paiement et partage de l'addition	42

4.13	Portail client : fidélité, points et récompenses	42
4.14	Back-office : gestion des ressources humaines	43
4.15	Organisation du déploiement Docker Compose	43
5.1	Sortie pytest backend : 335 tests réussis	47
5.2	Sorties Vitest : back-office et portail client réussis	47
5.3	Sortie Playwright client : 79 scénarios réussis et un sélecteur à corriger . .	48
5.4	Workflow CI et synthèse des validations locales	48

Liste des tableaux

1.1	Comparaison synthétique des solutions existantes	3
1.2	Périmètre fonctionnel confirmé	5
1.3	Planification pédagogique du projet	6
1.4	Méthodologie suivie pendant le projet	6
1.5	Répartition synthétique des contributions	7
2.1	Acteurs et responsabilités	8
2.2	Besoins fonctionnels principaux	9
2.3	Besoins non fonctionnels	10
2.4	Synthèse du modèle logique de données	16
3.1	Données utilisées par le pipeline de sentiment	23
3.2	Normalisation des sorties de sentiment	25
3.3	Comparaison des approches de sentiment	26
3.4	Détail des prédictions sur le corpus de validation	28
3.7	Synthèse comparative des métriques de performance	29
4.1	Stack technique confirmée	32
4.2	Modules backend réalisés	33
4.3	Applications frontend	35
5.1	Familles de tests dans le projet	45
5.2	Résultats de tests du 17 juin 2026	46
5.3	Exemples de scénarios backend critiques	49
5.4	Scénarios frontend validés	50

Table des matières

Dédicaces	i
Remerciements	ii
Résumé	iii
Liste des abréviations	iv
Liste des figures	vi
Liste des tableaux	vii
Introduction générale	1
1 Contexte général	2
Introduction	2
1.1 Contexte métier	2
1.2 Problématique	2
1.3 Étude de l'existant	3
1.4 Objectifs du projet	4
1.5 Périmètre fonctionnel retenu	4
1.6 Planification	5
1.7 Méthodologie de travail	6
1.8 Répartition du travail	7
Conclusion	7
2 Analyse des besoins et conception	8

2.1	Acteurs du système	8
2.2	Besoins fonctionnels	8
2.3	Besoins non fonctionnels	9
2.4	Diagrammes de cas d'utilisation	10
2.5	Diagramme de classes	12
2.6	Diagrammes de séquence	14
2.6.1	Authentification staff	14
2.6.2	Prise de commande et envoi vers le KDS	14
2.6.3	Création d'un avis client et analyse de sentiment	15
2.7	Modèle logique de données	16
2.8	Règles métier principales	17
2.9	Architecture fonctionnelle cible	17
	Conclusion	18
3	Analyse de sentiment et choix du modèle	19
	Introduction	19
3.1	Fondements théoriques	19
3.1.1	Intelligence artificielle, Machine Learning et Deep Learning	19
3.1.2	Sentiments et opinions	20
3.1.3	Traitement du langage naturel (NLP)	20
3.1.4	Approches de classification de sentiment	20
3.2	Procédure de traitement des avis textuels	21
3.2.1	Collecte de données et prétraitement	21
3.2.2	Vectorisation du texte (Word Embedding)	21
3.2.3	Prédiction de la polarité	22
3.3	Rôle de l'analyse de sentiment dans Tastify	22
3.4	Données utilisées	23
3.5	Pipeline technique	23
3.6	Modèle utilisé dans Tastify	24
3.7	Conversion des sorties	25

3.8	Comparaison avec des alternatives	25
3.9	Justification du choix	26
3.10	Évaluation expérimentale et métriques	27
3.10.1	Protocole d'évaluation et corpus	27
3.10.2	Analyse des erreurs et métriques	28
3.11	Limites de la fonctionnalité	30
	Conclusion	30
4	Architecture technique et réalisation	31
	Introduction	31
4.1	Stack technique	31
4.2	Architecture globale	32
4.3	Organisation du backend	33
4.4	Authentification et sécurité	33
4.5	Temps réel et Kitchen Display System	34
4.6	Applications frontend	35
4.7	Paieement et fidélité	35
4.8	Stocks et prévision simple	35
4.9	Moteur de recommandation de plats	36
4.10	Interfaces réalisées	36
4.11	Déploiement Docker Compose	43
	Conclusion	44
5	Tests, validation, limites et perspectives	45
	Introduction	45
5.1	Stratégie de tests	45
5.2	Résultats exécutés	46
5.3	Tests backend	49
5.4	Tests frontend et end-to-end	49
5.5	Validation de l'analyse de sentiment	50
5.6	Tests de charge	50

5.7	Limites du projet	51
5.8	Perspectives	51
	Conclusion	51
	Conclusion générale	52
	Bibliographie	53
A	Annexes	55
A.1	Commandes de déploiement et de test	55
A.1.1	Lancement et tests de la codebase	55

Introduction générale

Dans un restaurant, une commande mal transmise peut suffire à créer du retard en cuisine, une erreur de plat ou une mauvaise expérience client. La gestion ne se limite donc pas à afficher un menu en ligne ou à accepter des réservations. Elle touche directement le travail quotidien : coordination entre la salle et la cuisine, disponibilité des plats, suivi des stocks, paiements, fidélité et retours clients.

Tastify part de ce besoin concret. Le projet propose une application web centralisée pour gérer un restaurant, avec deux espaces distincts : une interface staff pour le gérant, les serveurs et la cuisine, puis un portail client pour consulter le menu, réserver, payer et laisser un avis. Sur le plan technique, la solution repose sur un backend Django REST Framework, une base MySQL, Redis et Celery pour le temps réel et les tâches asynchrones, Django Channels pour les WebSocket, et deux applications React.

Le périmètre du projet ne se limite pas à un menu numérique. Tastify relie les tables, les commandes, le Kitchen Display System, les stocks, les réservations, les paiements, la fidélité, les avis clients et les tableaux de bord. Ce choix rend le projet intéressant dans le cadre d'un Projet de Fin d'Année, car il demande de traiter plusieurs aspects du génie logiciel : modélisation, architecture, API, sécurité, temps réel, tâches de fond, interface utilisateur et validation.

Ce rapport présente Tastify selon une progression allant du besoin métier jusqu'à la conception, la réalisation, les tests et les perspectives d'amélioration.

Chapitre 1

Contexte général

Introduction

Ce chapitre place Tastify dans son contexte métier. Il décrit les problèmes rencontrés dans la gestion d'un restaurant, formule la problématique, compare quelques solutions existantes et fixe les objectifs retenus pour le projet.

1.1 Contexte métier

Un restaurant fonctionne avec des enchaînements rapides. Une table se libère, un serveur prend une commande, la cuisine prépare les plats, le stock diminue, le paiement est enregistré, puis le client peut laisser un avis. Chaque étape dépend de la précédente, et plusieurs rôles interviennent en même temps : gérant, serveur, cuisinier et client.

Quand ces informations sont réparties entre plusieurs outils, les erreurs arrivent vite. Un serveur peut annoncer un plat qui n'est plus disponible. La cuisine peut recevoir une commande incomplète. Le gérant peut découvrir une rupture de stock après le service. Les avis clients peuvent rester dispersés et ne servir à aucune décision. Une application utile doit donc réunir ces informations au même endroit et réduire les pertes de temps entre les acteurs.

1.2 Problématique

La problématique principale du projet peut être formulée de cette manière :

Comment concevoir une application web centralisée, maintenable et accessible permettant de gérer les opérations d'un restaurant, tout en améliorant la

coordination entre la salle, la cuisine, la gestion administrative et l'expérience client ?

Cette problématique conduit à plusieurs questions concrètes :

- comment séparer les accès selon les rôles sans multiplier les applications inutilement ;
- comment envoyer rapidement les commandes vers la cuisine ;
- comment relier commandes, stocks, paiements et fidélité ;
- comment analyser les avis clients sans ralentir l'expérience utilisateur ;
- comment garder une architecture claire, testable et simple à déployer.

1.3 Étude de l'existant

Le marché propose déjà plusieurs solutions de caisse ou de gestion de restaurant. Elles couvrent souvent une partie du besoin : prise de commande, paiement, gestion des stocks ou reporting. Le tableau 1.1 résume leur positionnement par rapport à Tastify.

TABLE 1.1 – Comparaison synthétique des solutions existantes

Solution	Points forts	Limites fréquentes	Lecture pour Tastify
Lightspeed Restaurant	Caisse, menu, commandes, reporting.	Coût récurrent, dépendance à une offre commerciale.	Confirme l'intérêt d'une solution intégrée.
Toast POS	Orientation restaurant, commande et paiement.	Disponibilité et adaptation au marché local variables.	Montre l'importance du lien salle-cuisine.
Revel Systems	Couverture ERP large, gestion multi-sites.	Coût et complexité élevés pour une petite structure.	Justifie un périmètre modulaire.
L'Addition	Solution spécialisée restauration, caisse et service.	Couverture variable selon les offres et les intégrations.	Sert de repère pour les besoins terrain.
Tastify	Projet web modulaire, deux portails, temps réel, avis et fidélité.	Prototype académique, pas de données de production.	Adapté au cadre pédagogique du PFA.

Cette comparaison montre que les solutions commerciales sont puissantes, mais qu’elles peuvent être coûteuses, fermées ou liées à un matériel précis. Tastify se place dans une logique pédagogique : comprendre les flux d’un restaurant, les modéliser proprement et proposer une architecture réutilisable.

1.4 Objectifs du projet

Les objectifs du projet sont les suivants :

- centraliser la gestion des menus, catégories, plats, tables, commandes et réservations ;
- proposer une interface staff adaptée au gérant, au serveur et au cuisinier ;
- proposer un portail client pour consulter le menu, réserver, payer et suivre la fidélité ;
- améliorer la coordination cuisine avec un Kitchen Display System et des WebSocket ;
- gérer les stocks, les ingrédients, les recettes et les mouvements associés ;
- analyser les avis clients avec un traitement de sentiment lancé en arrière-plan ;
- obtenir une architecture testable, documentée et déployable avec Docker Compose.

1.5 Périmètre fonctionnel retenu

L’implémentation couvre les modules listés dans le tableau 1.2.

TABLE 1.2 – Périmètre fonctionnel confirmé

Domaine	Fonctions couvertes
Menu	Catégories, plats, images, disponibilité et liens avec les ingrédients.
Salle	Tables, plan de salle, statut de table et prise de commande.
Cuisine	KDS, lignes de commande, statuts de préparation et synchronisation temps réel.
Stock	Ingrédients, seuils d’alerte, recettes et mouvements de stock.
RH	Employés, shifts, offres d’emploi et candidatures.
Réservations	Création, disponibilité, conflits de créneaux et gestion côté client/staff.
Paielements	Paieement d’une commande, contribution par ligne et lien avec le client.
Fidélité	Profil, points, transactions et récompenses.
Avis	Commentaires, note optionnelle et analyse de sentiment.
Analytics	Indicateurs de tableau de bord et statistiques de sentiment.
Configuration	Informations du restaurant, devise, couleurs, paramètres de service.

1.6 Planification

La planification ci-dessous synthétise les principales phases suivies durant le projet, à partir des modules livrés, des étapes de conception, des développements réalisés et des validations effectuées.

TABLE 1.3 – Planification pédagogique du projet

Phase	Contenu	Durée indicative
Phase 1	Initialisation du dépôt, environnement Docker, modèles principaux, authentification et rôles.	2 semaines
Phase 2	Modules gérant : menu, catégories, stocks, ressources humaines et configuration.	2 semaines
Phase 3	Salle et cuisine : tables, commandes, KDS, statuts et WebSocket.	2 semaines
Phase 4	Portail client : menu public, réservation, compte, panier et paiement.	2 semaines
Phase 5	Fidélité, avis, analyse de sentiment et indicateurs analytics.	2 semaines
Phase 6	Tests, documentation, conteneurisation et préparation du rapport.	1 semaine

1.7 Méthodologie de travail

Le projet a été conduit par itérations successives afin de garder un lien constant entre les besoins métier, la conception et l'implémentation. Chaque phase a produit un résultat vérifiable : modèle de données, API, écran React, scénario de test ou élément documentaire.

TABLE 1.4 – Méthodologie suivie pendant le projet

Phase	Travail réalisé
Cadrage	Analyse du besoin restaurant, identification des acteurs, choix du périmètre et comparaison avec des solutions existantes.
Conception	Modélisation UML, découpage des modules Django, définition des routes API et organisation des deux applications React.
Réalisation backend	Implémentation des modèles, serializers, vues DRF, permissions, WebSocket, tâches Celery et services métier.
Réalisation frontend	Construction du back-office staff et du portail client avec React, Vite, TypeScript et Tailwind CSS.
Intégration	Connexion des frontends aux API, orchestration Docker Compose, synchronisation salle-cuisine et validation des flux de bout en bout.
Validation	Exécution des tests pytest, Vitest et Playwright, capture des interfaces, correction documentaire et préparation du rapport final.

Les outils utilisés ont structuré le travail quotidien : Git pour suivre l'évolution du code, Docker Compose pour lancer l'environnement complet, Django REST Framework pour l'API, React et Vite pour les interfaces, MySQL pour la persistance, Redis et Celery pour le temps réel et les traitements asynchrones, puis pytest, Vitest et Playwright pour la validation.

Les principales difficultés ont concerné la séparation claire des rôles, la synchronisation entre la salle et la cuisine, la cohérence entre paiements, commandes et fidélité, ainsi que la production d'un rapport fidèle au projet réel. Ces points ont été traités par un découpage modulaire, des tests ciblés et une documentation progressive des décisions.

1.8 Répartition du travail

La répartition a combiné spécialisation technique et validation croisée. Mehdi s'est concentré sur le socle backend, l'intégration, les tests et le déploiement. Ibtiha1 s'est concentrée sur les interfaces, l'expérience utilisateur, la documentation, les diagrammes UML et la validation fonctionnelle.

TABLE 1.5 – Répartition synthétique des contributions

Membre	Tâches réalisées	Contribution principale
Mehdi	Backend Django/DRF, modèles métier, API, authentification JWT, permissions, Redis, Celery, Docker Compose, tests pytest, intégration et préparation des résultats de validation.	Fiabilisation du cœur applicatif, cohérence des modules métier et validation technique.
Ibtiha1	Frontends React staff et client, parcours utilisateur, interfaces UI, documentation du rapport, diagrammes UML, captures fonctionnelles et validation des scénarios métier.	Qualité de l'expérience utilisateur, lisibilité documentaire et cohérence fonctionnelle.

Conclusion

Le contexte étudié montre l'intérêt d'une application centralisée pour un restaurant. Tastify répond à ce besoin avec une architecture full-stack qui couvre les flux métier principaux. Le chapitre suivant détaille les acteurs, les besoins et la conception du système.

Chapitre 2

Analyse des besoins et conception

Ce chapitre précise les besoins de Tastify et présente la conception fonctionnelle et objet. La modélisation s’appuie sur UML, langage standard pour représenter les systèmes logiciels [3]. Les diagrammes présentés décrivent les cas d’utilisation, la structure de données et les interactions dynamiques du projet.

2.1 Acteurs du système

Tastify distingue quatre acteurs fonctionnels. Les accès dépendent du rôle associé à chaque utilisateur.

TABLE 2.1 – Acteurs et responsabilités

Acteur	Responsabilités principales
Gérant	Gérer le restaurant, gérer le menu, les stocks, les employés, les paramètres, les réservations, les avis et les indicateurs.
Serveur	Consulter le plan de salle, créer les commandes, suivre les tables et déclencher l’encaissement.
Cuisinier	Consulter les commandes cuisine, mettre à jour les statuts de préparation et signaler les plats prêts.
Client	Consulter le menu, s’inscrire, se connecter, réserver, payer, consulter sa fidélité et laisser un avis.

2.2 Besoins fonctionnels

Les besoins fonctionnels sont regroupés par domaine métier dans le tableau 2.2.

TABLE 2.2: Besoins fonctionnels principaux

Module	Besoins
Authentification	Inscription et connexion, séparation des portails staff/client, jetons JWT, refresh par cookie, contrôle des accès par rôle.
Menu	Créer, modifier, masquer ou afficher des catégories et des plats ; gérer la disponibilité et les images.
Salle	Visualiser les tables, leur capacité et leur statut ; accéder à la prise de commande par table.
Commandes	Créer des commandes, enregistrer les lignes, figer les prix unitaires et suivre les statuts de préparation.
Cuisine	Afficher les commandes dans un KDS et synchroniser les changements de statut vers le staff.
Stock	Gérer les ingrédients, les seuils d'alerte, les recettes et les mouvements de stock.
Réservations	Permettre au client de réserver et au staff de suivre les demandes selon la disponibilité.
Paielements	Enregistrer les paiements, gérer les méthodes disponibles et associer des contributions aux lignes de commande.
Fidélité	Suivre les points, transactions, paliers et récompenses clients.
Avis	Collecter les commentaires, stocker une note optionnelle et déclencher l'analyse de sentiment.
Analytics	Agréger des indicateurs : revenus, commandes, plats populaires, stocks et sentiment client.
Configuration	Paramétrer les informations du restaurant, la devise, certains réglages de service et l'identité visuelle.

2.3 Besoins non fonctionnels

Tastify doit aussi respecter plusieurs contraintes techniques pour rester maintenable.

TABLE 2.3 – Besoins non fonctionnels

Besoin	Traduction dans le projet
Sécurité	Authentification JWT, permissions DRF, rôles gérant/serveur/cuisinier/client, séparation des portails.
Maintenabilité	Découpage en applications Django spécialisées et deux frontends React séparés.
Réactivité	WebSocket pour les événements staff et KDS ; tâches Celery pour les traitements non bloquants.
Traçabilité	Modèles avec dates de création et de mise à jour ; conservation des prix de commande.
Déploiement	Docker Compose avec services backend, base de données, Redis, worker, beat et frontends.
Testabilité	Tests backend avec pytest, tests frontend avec Vitest et scénarios end-to-end avec Playwright.

2.4 Diagrammes de cas d'utilisation

Les diagrammes de cas d'utilisation décrivent les échanges principaux entre les acteurs et les modules.

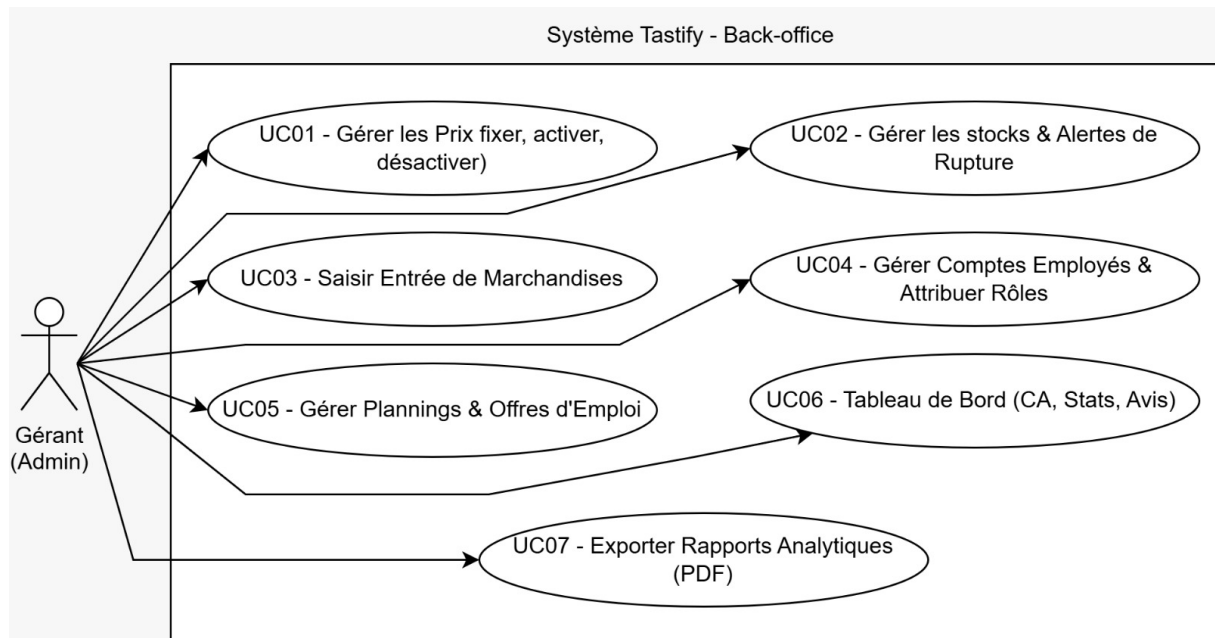


FIGURE 2.1 – Diagramme de cas d'utilisation du back-office gérant

La figure 2.1 regroupe les fonctions d'administration : menu, stock, employés, alertes,

tableaux de bord et rapports.

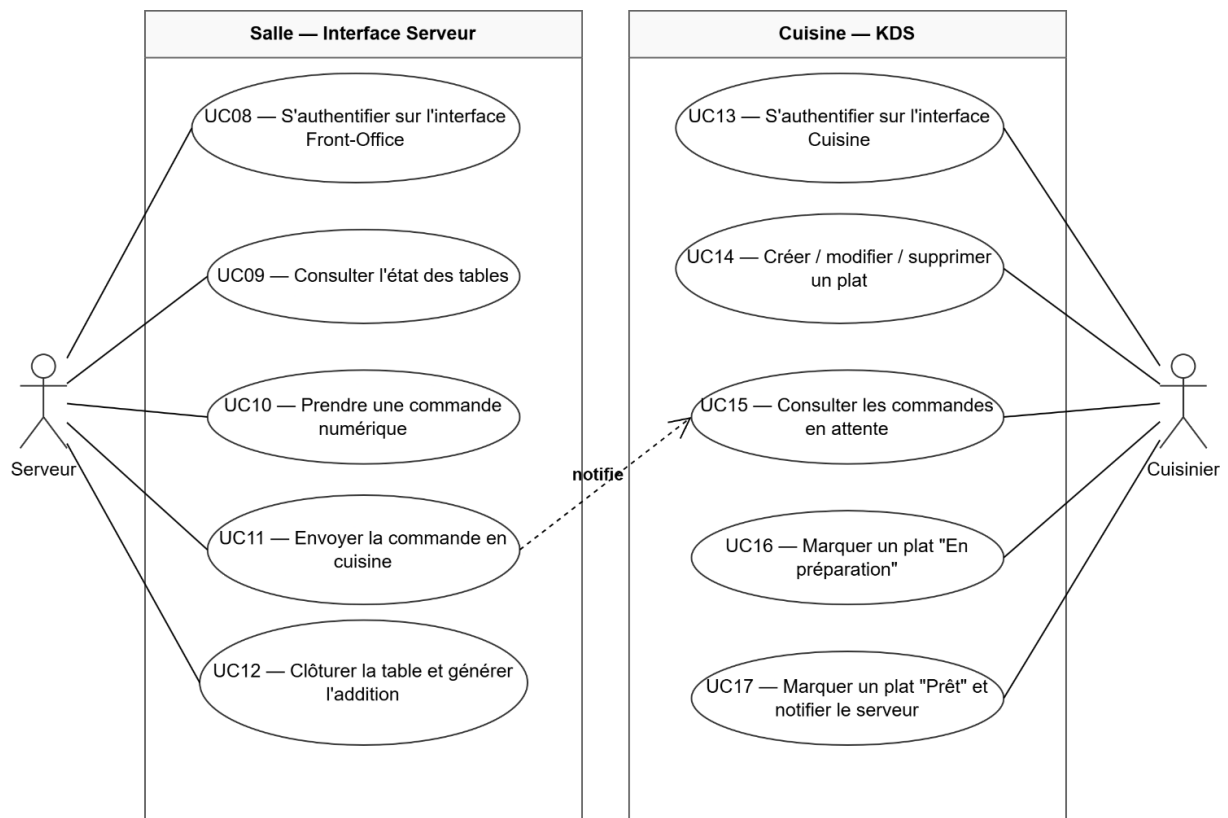


FIGURE 2.2 – Diagramme de cas d'utilisation des modules salle et cuisine

La figure 2.2 montre le lien entre le serveur, la salle et le KDS. Elle explique le besoin de temps réel : la commande doit passer de l'interface serveur vers l'écran cuisine sans attente inutile.

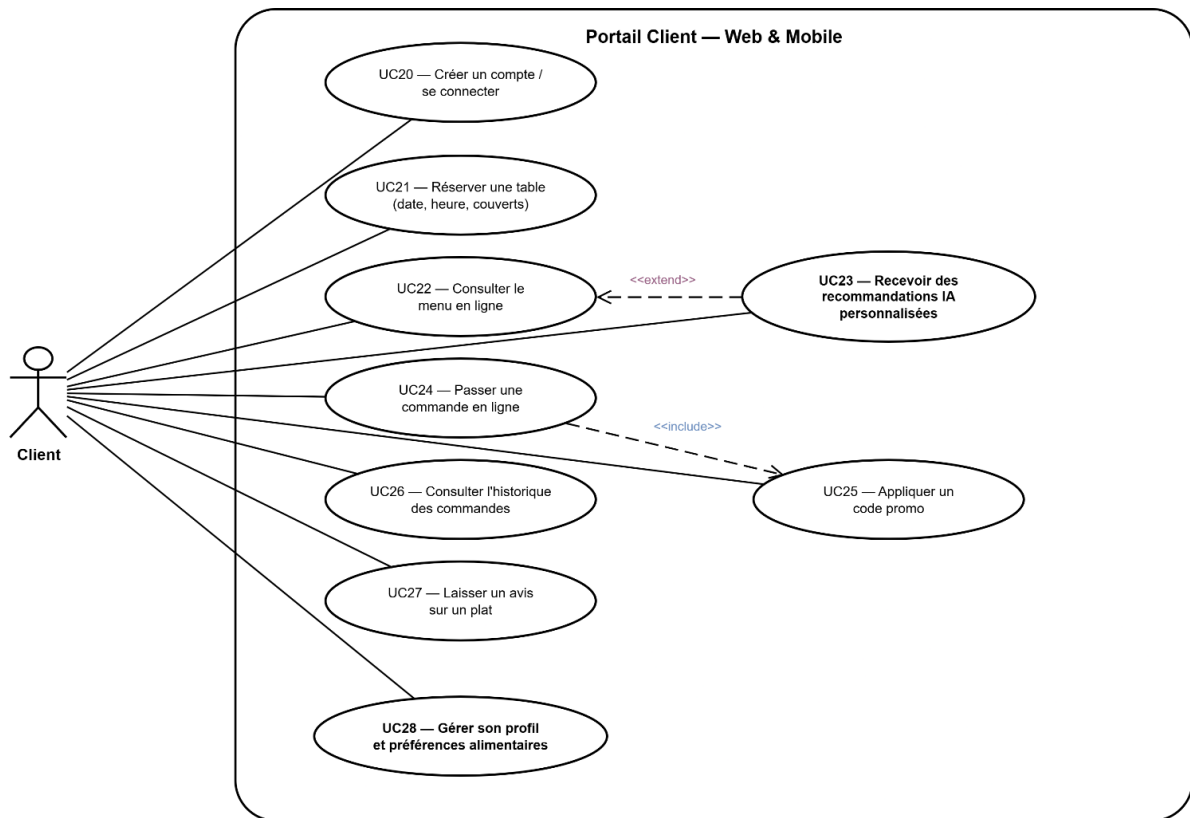


FIGURE 2.3 – Diagramme de cas d'utilisation du portail client

La figure 2.3 présente les fonctions disponibles côté client : consultation du menu, réservation, commande, paiement, fidélité et avis.

2.5 Diagramme de classes

Le diagramme de classes présente les entités majeures et leurs relations.

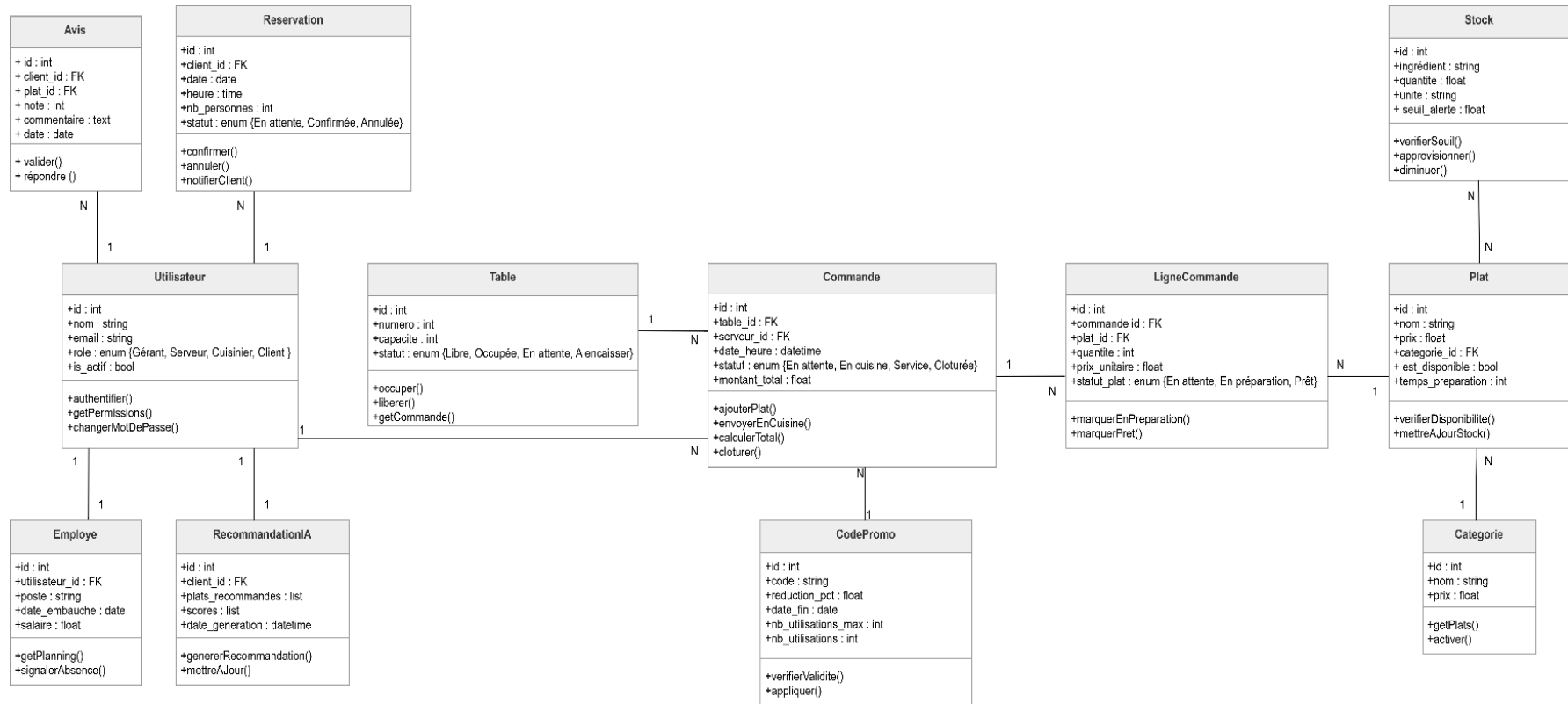


FIGURE 2.4 – Diagramme de classes de Tastify

La figure 2.4 correspond aux domaines présents dans l'application : utilisateurs, plats, tables, commandes, réservations, paiements, avis et stock. Certaines entités détaillées dans le code ne sont pas toutes visibles sur le schéma structurel initial ; elles sont donc précisées dans le modèle logique ci-dessous.

2.6 Diagrammes de séquence

Afin d'illustrer la dynamique des échanges et le comportement du système lors de scénarios clés, trois diagrammes de séquence sont introduits ci-dessous.

2.6.1 Authentification staff

Le processus d'authentification sécurisé pour les membres du personnel repose sur des jetons JWT. Le serveur, le cuisinier ou le gérant fournit ses identifiants qui sont validés par l'API backend avant d'émettre les clés de session, comme le montre la figure 2.5.

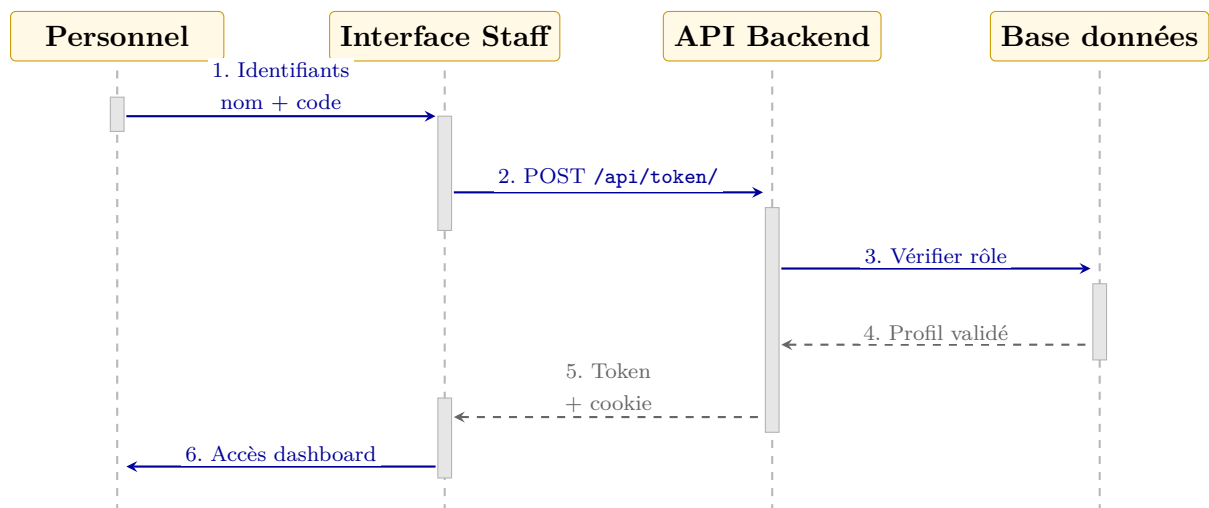


FIGURE 2.5 – Diagramme de séquence de l'authentification staff

La figure 2.5 détaille la séparation entre la saisie côté interface, la validation par l'API et la vérification du rôle en base de données avant l'accès au tableau de bord.

2.6.2 Prise de commande et envoi vers le KDS

Ce scénario décrit le flux de traitement en temps réel depuis la prise de commande par le serveur en salle jusqu'à sa diffusion instantanée vers l'écran de cuisine KDS grâce au protocole WebSocket géré par Redis, comme le montre la figure 2.6.

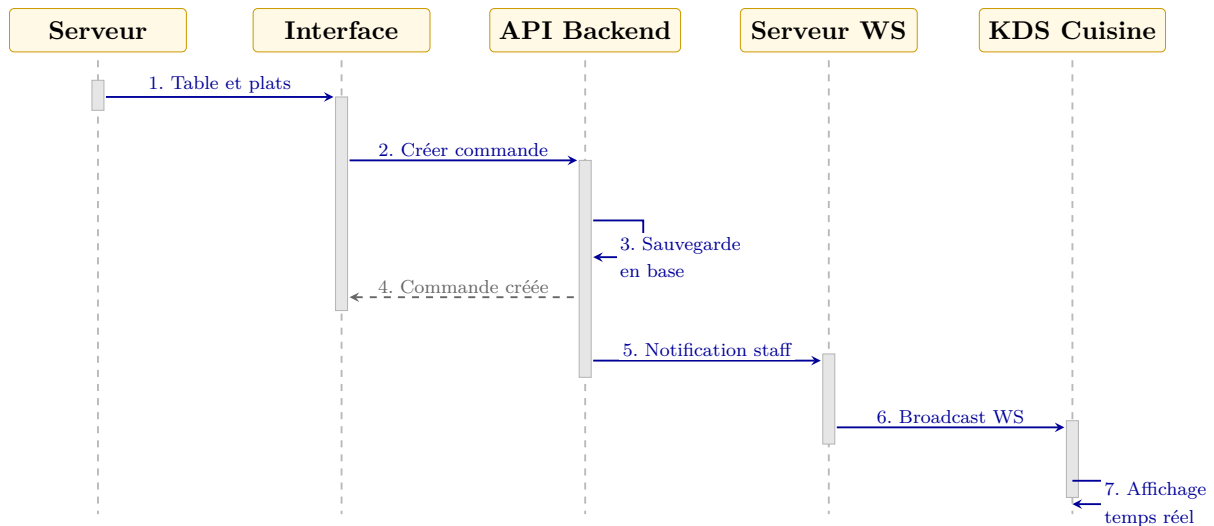


FIGURE 2.6 – Diagramme de séquence de la prise de commande et envoi au KDS

La figure 2.6 met en évidence le passage de la commande depuis l'interface staff vers l'API, puis la notification WebSocket qui permet au KDS cuisine de se mettre à jour en temps réel.

2.6.3 Création d'un avis client et analyse de sentiment

Le client soumet son commentaire et sa note depuis le portail public. Le backend enregistre l'avis et délègue en tâche d'arrière-plan Celery l'appel à l'API de traitement de langage naturel de Hugging Face pour classer le sentiment, comme le montre la figure 2.7.

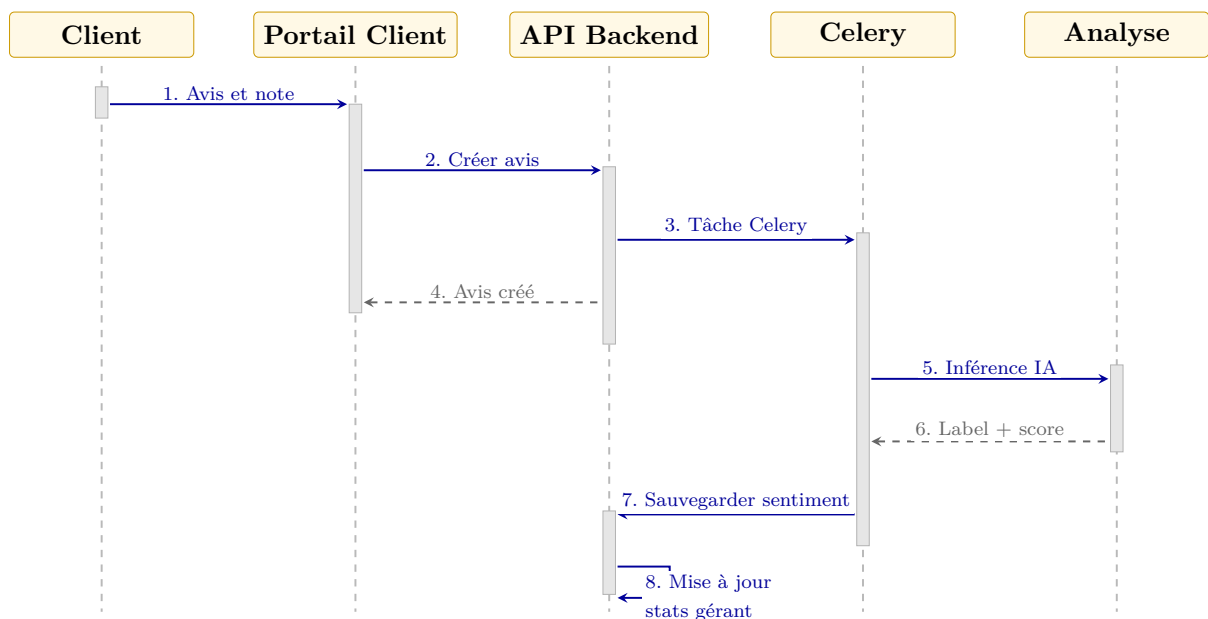


FIGURE 2.7 – Diagramme de séquence de la création d'un avis et analyse de sentiment

La figure 2.7 montre que la création de l'avis reste rapide pour le client, tandis que l'analyse est exécutée ensuite par Celery puis sauvegardée pour alimenter les statistiques.

2.7 Modèle logique de données

Le modèle logique de données est construit à partir des modèles Django de la codebase. Le tableau 2.4 ne liste pas toutes les colonnes techniques. Il résume les tables structurantes et leurs relations.

TABLE 2.4: Synthèse du modèle logique de données

Table / entité	Relations principales	Rôle
Utilisateur	Rôle applicatif.	Compte de connexion et séparation gérant, serveur, cuisinier, client.
Categorie, Plat	Catégorie 1..N plats.	Structure de la carte, prix, disponibilité, image et temps de préparation.
Ingredient, PlatIngredient	Relation N..M entre plats et ingrédients.	Recettes et quantités requises pour relier menu et stock.
Table, PlanText	Table liée aux commandes et réservations.	Plan de salle, capacité, position et statut opérationnel.
Commande, CommandeLigne	Commande 1..N lignes ; table, serveur ou client optionnels.	Prise de commande, suivi cuisine, prix figé et total.
Reservation	Client, table, date et créneau.	Réservation avec statut et contrôle de disponibilité.
Paielement, PaiementItem	Paielement lié à une commande ; contributions liées aux lignes.	Encaissement et partage possible de l'addition.
Avis, Analyse Sentiment	Avis 1..1 analyse ; avis lié à plat ou commande.	Commentaire client, note optionnelle, label et score de sentiment.
LoyaltyProfile, Reward	Profil 1..1 utilisateur ; transactions et récompenses.	Programme de fidélité.

Table / entité	Relations principales	Rôle
WeatherData	Données datées.	Support aux indicateurs et prévisions simples.
Restaurant Configuration	Configuration unique.	Paramètres du restaurant, identité, devise et réglages de service.

2.8 Règles métier principales

Les règles suivantes structurent le comportement attendu :

- un utilisateur possède un rôle qui détermine les routes et les actions accessibles ;
- une commande peut être liée à une table, à un serveur et parfois à un client ;
- chaque ligne de commande conserve le prix unitaire du plat au moment de la commande ;
- les statuts de lignes pilotent le KDS : attente, préparation, prêt, servi ou annulé ;
- un paiement est toujours lié à une commande et peut comporter des contributions par ligne ;
- un avis appartient à un utilisateur et peut être lié à un plat ou à une commande ;
- une analyse de sentiment est associée à un seul avis ;
- le stock repose sur les ingrédients, les seuils d’alerte et les mouvements.

2.9 Architecture fonctionnelle cible

La conception sépare deux usages : une application staff pour les rôles internes et un portail client. Les deux utilisent le même backend API.

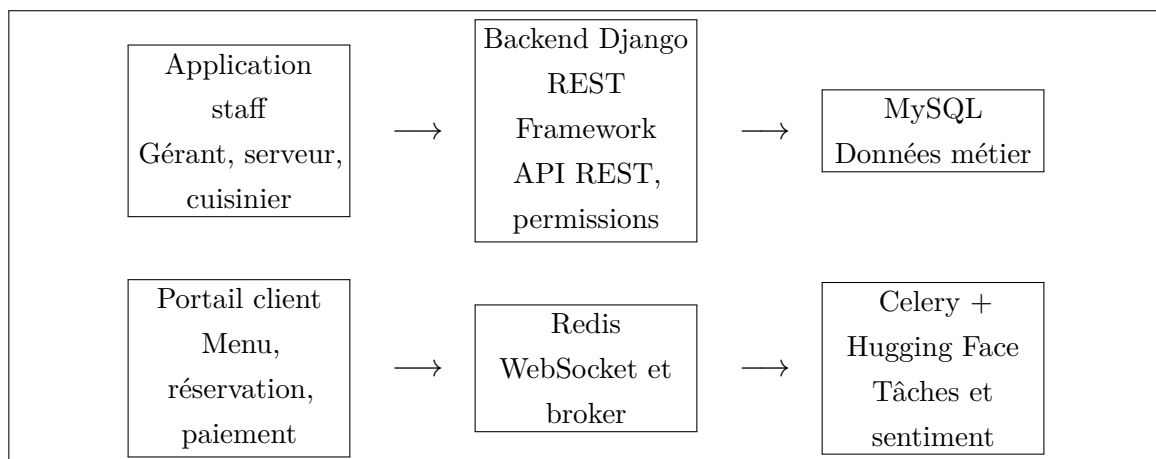


FIGURE 2.8 – Architecture fonctionnelle générale de Tastify

La figure 2.8 résume le choix principal du projet : un backend commun expose les services, tandis que les interfaces restent séparées selon les usages.

Conclusion

L'analyse des besoins fait apparaître un système multi-acteurs, organisé autour d'un backend commun et de deux interfaces web. Les diagrammes existants donnent une base UML, complétée par le modèle logique tiré du code. Le chapitre suivant détaille l'analyse de sentiment, utilisée pour traiter les retours clients.

Chapitre 3

Analyse de sentiment et choix du modèle

Introduction

L'analyse de sentiment est la partie d'intelligence applicative la plus visible dans Tastify. Elle ne remplace pas le jugement du gérant, mais elle transforme les avis textuels en indicateurs simples : sentiment positif, neutre ou négatif, score de confiance et modèle utilisé. Ce chapitre détaille d'abord le cadre théorique qui sous-tend ce traitement avant de décrire son intégration technique et de présenter une évaluation quantitative de ses performances.

3.1 Fondements théoriques

3.1.1 Intelligence artificielle, Machine Learning et Deep Learning

L'intelligence artificielle (IA) englobe l'ensemble des techniques permettant à des machines de simuler des processus cognitifs humains. Au sein de l'IA, l'apprentissage automatique (*Machine Learning* ou ML) désigne l'approche par laquelle un algorithme apprend à réaliser une tâche à partir de données d'entraînement, sans programmation explicite des règles de décision. Enfin, l'apprentissage profond (*Deep Learning* ou DL) représente une sous-catégorie du ML reposant sur des réseaux de neurones artificiels profonds, capables d'extraire automatiquement des représentations hiérarchiques et complexes à partir de données brutes.

Dans notre projet, l'IA joue un rôle d'aide à la décision opérationnelle pour le gérant : elle automatise le suivi de la qualité de service en extrayant la polarité des retours textuels

rédigés par les clients.

3.1.2 Sentiments et opinions

Une opinion est une évaluation subjective formulée par un individu sur une entité (un plat, un service, un restaurant). Elle se distingue d'un simple sentiment par sa structure formelle. En recherche académique, une opinion est généralement représentée sous la forme d'un quintuple (e, a, s, h, t) où :

- e est l'entité cible (ex. : **Plat** : **Tagine**);
- a est l'aspect spécifique évalué (ex. : **Goût**, **Température**);
- s est la polarité ou le score du sentiment (positif, neutre, négatif);
- h est le détenteur de l'opinion (*opinion holder*, le client);
- t est le repère temporel (date et heure de l'avis).

L'analyse de sentiment dans Tastify vise à automatiser l'identification de la polarité s du commentaire saisi, pour l'associer à l'avis global du client.

3.1.3 Traitement du langage naturel (NLP)

Le traitement du langage naturel (*Natural Language Processing* ou NLP) est le domaine de l'IA dédié à la compréhension, la génération et la manipulation du langage humain par les ordinateurs. L'analyse de sentiment est une tâche classique de NLP, formulée comme un problème de classification de textes où un modèle prend en entrée une suite de caractères et renvoie une probabilité d'appartenance à des classes de polarité.

Les niveaux d'analyse de sentiment incluent l'analyse au niveau du document entier, au niveau de la phrase individuelle, ou au niveau de l'aspect (identifier le sentiment pour chaque ingrédient ou service mentionné). Tastify réalise une analyse combinée au niveau du document (le commentaire de l'avis) et l'associe de façon optionnelle à un plat spécifique.

3.1.4 Approches de classification de sentiment

Les approches de classification de sentiment étudiées pour cadrer le choix technique du projet se répartissent en trois catégories principales :

1. **Les approches lexicales (basées sur des règles)** : Elles utilisent des dictionnaires de mots annotés (comme SentiWordNet) et comptent les occurrences. Simples et transparentes, elles manquent de flexibilité et échouent face à la négation, aux sarcasmes et à l'évolution du langage.
2. **Les approches de Machine Learning classique** : Des algorithmes supervisés comme les machines à vecteurs de support (SVM) ou le classifieur Bayésien Naïf

(*Naive Bayes*) s'entraînent sur des vecteurs de mots. Ils sont performants mais exigent une ingénierie de caractéristiques (*feature engineering*) manuelle.

3. **Les approches basées sur le Deep Learning et les Transformers** : Les architectures récentes comme BERT (*Bidirectional Encoder Representations from Transformers*) capturent les relations bidirectionnelles entre les mots grâce à des mécanismes d'attention. Elles atteignent l'état de l'art mondial en comprenant le contexte complet d'une phrase.

3.2 Procédure de traitement des avis textuels

Le traitement du texte brut suit une séquence d'étapes rigoureuses pour transformer les caractères en données exploitables par nos modèles de classification.

3.2.1 Collecte de données et prétraitement

Lorsqu'un client saisit un avis, le texte brut est récupéré via l'API. Avant l'inférence, plusieurs traitements standard de nettoyage sont appliqués :

- **Nettoyage et normalisation** : Passage en minuscules, suppression des caractères spéciaux, des emojis non textuels et des balises HTML.
- **Tokenisation** : Découpage de la chaîne de caractères en unités linguistiques élémentaires nommées *tokens* (mots ou sous-mots).
- **Suppression des mots vides (*Stop Words*)** : Retrait des mots très fréquents mais dénués de valeur sémantique pour la polarité (ex. : *le, la, un, de, et, is, the*).
- **Stemming (Racinement) / Lemmatisation** : Réduction des mots à leur racine (ex. : *mangeait, manger, mangent* deviennent *mang-*) ou à leur forme canonique de dictionnaire (*manger*) pour réduire la dimensionnalité.

3.2.2 Vectorisation du texte (Word Embedding)

Pour qu'un modèle mathématique traite le texte, celui-ci doit être converti en vecteurs numériques. Les techniques étudiées incluent :

- **TF-IDF (Term Frequency - Inverse Document Frequency)** : Une méthode statistique calculant l'importance d'un mot t dans un document d par rapport à un corpus D . Les formules associées sont :

$$TF(t, d) = \frac{f_{t,d}}{\sum_{t'} f_{t',d}}$$

$$IDF(t, D) = \log \left(\frac{|D|}{|\{d \in D : t \in d\}|} \right)$$

$$TF-IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

- **Word2Vec** : Apprentissage de représentations denses basées sur le contexte local (modèles CBOW ou Skip-gram).
- **FastText** : Extension de Word2Vec prenant en compte les n-grammes de caractères, facilitant le traitement des mots hors vocabulaire.
- **GloVe** : Factorisation globale de la matrice de co-occurrence des mots sur l'ensemble du corpus.
- **BERT** : Modèle pré-entraîné bidirectionnel qui génère des représentations vectorielles dynamiques dépendantes du contexte de chaque mot dans la phrase.

Dans Tastify, la tokenisation et la vectorisation sont encapsulées directement dans le modèle BERT de Hugging Face lors de l'inférence, tandis que notre fallback local applique une recherche de sous-chaînes optimisée.

3.2.3 Prédiction de la polarité

Les vecteurs sémantiques sont ensuite utilisés pour prédire la polarité du commentaire. Les méthodes traditionnelles incluent des règles simples ou des classifieurs linéaires de Machine Learning. Dans le cas du modèle principal de Tastify, le vecteur sémantique de sortie du modèle BERT passe par une couche de classification (Softmax) afin de produire une prédiction de sentiment, exprimée sous forme de probabilités pour les classes positif, neutre et négatif.

3.3 Rôle de l'analyse de sentiment dans Tastify

Dans Tastify, les avis clients donnent un retour direct sur l'expérience au restaurant. Un commentaire peut signaler un plat apprécié, un service lent, une attente trop longue, une mauvaise température ou une satisfaction générale. Sans traitement automatique, ces retours deviennent difficiles à suivre dès que leur nombre augmente.

L'analyse de sentiment joue donc deux rôles :

- **opérationnel**, car elle aide le gérant à repérer rapidement les avis défavorables ;
- **analytique**, car elle alimente les statistiques de satisfaction affichées dans le tableau de bord.

L'implémentation couvre les statistiques de sentiment dans les indicateurs. Le résultat de l'analyse est donc exploité par l'application, il n'est pas seulement stocké.

3.4 Données utilisées

Tastify utilise les avis saisis par les utilisateurs de l'application. Le tableau 3.1 décrit les informations exploitées.

TABLE 3.1 – Données utilisées par le pipeline de sentiment

Champ	Utilisation
<code>Avis.commentaire</code>	Texte libre analysé par le pipeline. C'est l'entrée principale du modèle.
<code>Avis.note</code>	Note optionnelle entre 1 et 5. Elle est stockée, mais le code d'analyse actuel se base sur le commentaire.
<code>Avis.lang_code</code>	Langue détectée simplement : ar si le texte contient des caractères arabes, en sinon.
<code>AnalyseSentiment.label</code>	Résultat normalisé : POSITIF , NEUTRE ou NEGATIF .
<code>AnalyseSentiment.score_brut</code>	Score de confiance renvoyé par Hugging Face ou par le fallback local.
<code>AnalyseSentiment.modele_utilise</code>	Nom du modèle distant ou de la méthode locale réellement utilisée.

Le texte envoyé à Hugging Face est limité à 512 caractères dans la tâche Celery. Ce choix suffit pour des avis courts et évite d'envoyer des commentaires trop longs au modèle.

3.5 Pipeline technique

Le pipeline démarre lors de la création d'un avis dans le `AvisViewSet`. L'avis est enregistré rapidement, puis l'analyse est confiée à une tâche Celery.

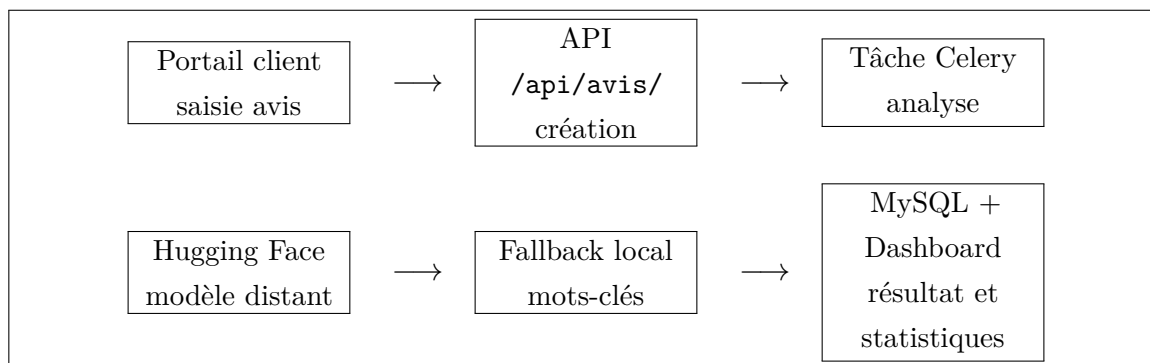


FIGURE 3.1 – Pipeline technique de l'analyse de sentiment

La figure 3.1 résume le flux : l'utilisateur soumet un avis, le backend l'enregistre,

Celery lance l'analyse, puis le résultat est stocké et agrégé dans les statistiques.

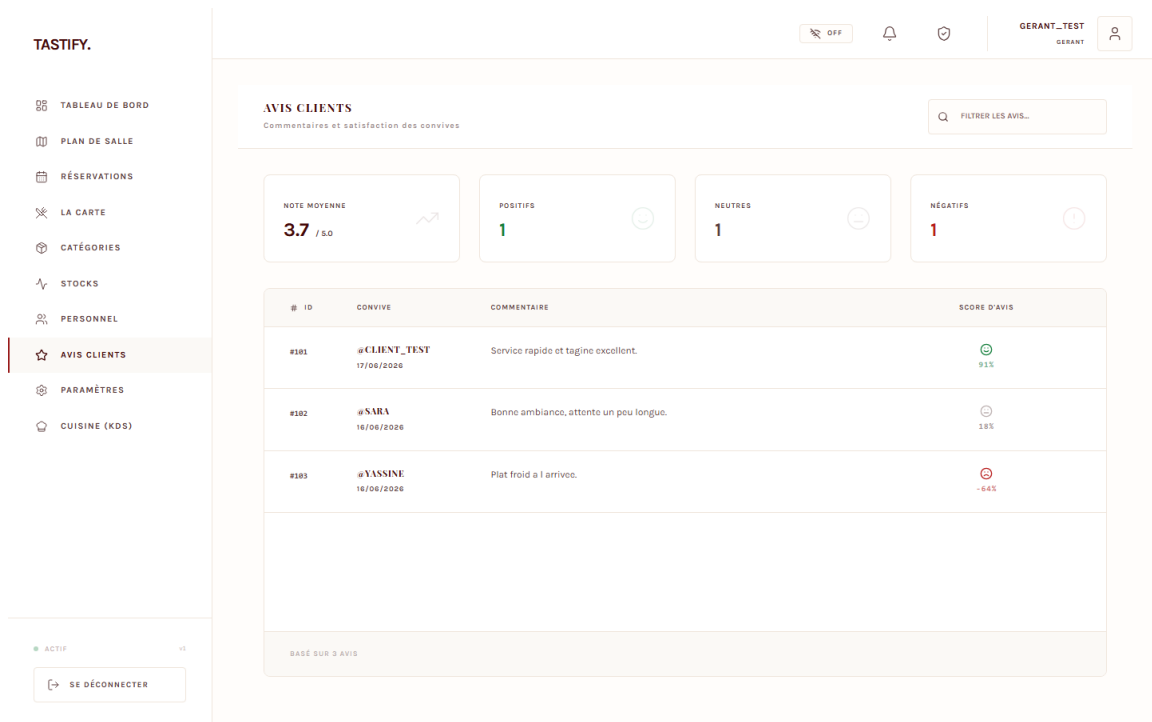


FIGURE 3.2 – Back-office : avis clients avec scores de sentiment

La figure 3.2 montre l'exploitation concrète du pipeline dans l'interface : chaque avis conserve son texte, sa note et un résultat de sentiment utilisable par le gérant pour prioriser les retours.

3.6 Modèle utilisé dans Tastify

Le code du fichier `apps/avis/tasks.py` définit les modèles suivants :

- `nlptown/bert-base-multilingual-uncased-sentiment` pour les avis non arabes ;
- `moussaKam/MARBERT-sentiment` pour les textes contenant des caractères arabes ;
- `nlptown/bert-base-multilingual-uncased-sentiment` comme modèle de secours si le modèle arabe ne répond pas ;
- `mots-cles-simple` comme secours local lorsque l'API Hugging Face n'est pas disponible ou lorsqu'aucun jeton n'est configuré.

Le modèle principal `nlptown/bert-base-multilingual-uncased-sentiment` est un BERT multilingue affiné pour l'analyse de sentiment sur des avis produits en six langues, dont le français [16]. Il renvoie une prédiction sous forme d'étoiles de 1 à 5. Le code convertit ensuite cette sortie en trois classes : négatif, neutre ou positif. Ce choix correspond bien à un restaurant, car les commentaires clients ressemblent souvent à des avis courts sur un produit ou un service.

La base scientifique de ce type de modèle vient de BERT, qui apprend des représentations contextuelles bidirectionnelles et peut être adapté à des tâches de classification avec une couche de sortie [7]. Pour les textes arabes, l'application utilise MARBERT, une famille de modèles conçus pour l'arabe et ses variantes dialectales [1].

3.7 Conversion des sorties

Les sorties du modèle ne sont pas exposées telles quelles au reste de l'application. Elles sont normalisées pour être lisibles dans le tableau de bord (Table 3.2).

TABLE 3.2 – Normalisation des sorties de sentiment

Sortie modèle	Label Tastify	Score métier
5 stars, stars, POSITIVE, LABEL_2	4 POSITIF	+15, avis favorable.
3 stars, NEUTRAL, LABEL_1	NEUTRE	0, signal sans polarité forte.
1 star, stars, NEGATIVE, LABEL_0	2 NEGATIF	−15, avis défavorable à surveiller.

Cette conversion rend l'indicateur plus lisible pour le gérant, tout en gardant dans `score_brut` la confiance renvoyée par le modèle.

3.8 Comparaison avec des alternatives

Le tableau 3.3 compare le modèle principal avec plusieurs alternatives possibles.

TABLE 3.3 – Comparaison des approches de sentiment

Approche	Statut Tastify	dans	Forces	Limites	Pertinence
BERT multilingue NLP Town	Modèle principal		Supporte le français, l'anglais, l'espagnol, l'italien, l'allemand et le néerlandais ; entraîné sur des avis ; utilisable via API [16, 12].	Moins spécialisé qu'un modèle français pur ; dépend d'un service externe si utilisé via API.	Très adapté aux avis courts de restaurant multilingues.
DistilCamemBERT sentiment	Alternative française		Spécialisé français ; modèle plus léger ; résultats documentés sur Amazon Reviews et Allociné [6, 14].	Ne couvre pas naturellement l'arabe ou l'anglais ; demande un routage par langue.	Bon choix si le périmètre devient surtout francophone.
XLM-R / XLM-T	Alternative multilingue		Bonne base de transfert multilingue ; XLM-R est conçu pour de nombreux langages [5] ; XLM-T vise les textes sociaux multilingues [2].	Demande un choix de modèle fine-tuné et des validations supplémentaires.	Intéressant pour une version multilingue plus avancée.
MARBERT sentiment	Modèle utilisé	arabe	Appuyé sur la famille MARBERT, adaptée aux variétés arabes et au texte social [1].	Spécifique aux textes arabes ; ne remplace pas le modèle multilingue pour le français.	Adapté aux avis en arabe ou en dialecte marocain.
Mots-clés / TF-IDF local	Fallback partiel		Fonctionne hors ligne, transparent et peu coûteux ; le fallback actuel par mots-clés est déjà implémenté.	Faible face à l'ironie, aux négations et au vocabulaire varié ; TF-IDF n'est pas implémenté.	Utile comme secours, mais insuffisant comme moteur principal.

Cette comparaison qualitative est complétée par l'évaluation expérimentale de la section 3.10, qui présente les prédictions détaillées et les métriques chiffrées obtenues sur un corpus de validation.

3.9 Justification du choix

Le choix du modèle `nlptown/bert-base-multilingual-uncased-sentiment` est motivé par sa proximité avec le domaine des avis clients, son excellent support de la langue française, sa rapidité d'intégration via l'API Hugging Face, et la garantie de continuité de service offerte par notre fallback local.

3.10 Évaluation expérimentale et métriques

3.10.1 Protocole d'évaluation et corpus

Afin d'évaluer de manière objective et honnête les performances de notre système sans disposer d'un grand corpus d'entraînement local, nous avons construit un jeu de validation composé de 15 avis réalistes rédigés en français et en arabe dialectal marocain (Darija), représentant les interactions courantes des clients en restaurant. Ces avis ont été annotés manuellement (5 Positifs, 5 Neutres, 5 Négatifs) pour constituer la vérité de terrain (*Ground Truth*). Ce corpus de validation est un échantillon réduit construit manuellement pour vérifier le comportement du pipeline sur des avis courts en français et en darija. Les labels de référence ont été attribués manuellement par les membres du projet. Cette évaluation reste indicative et ne remplace pas une validation sur un corpus réel de grande taille.

Le tableau 3.4 présente les avis de ce corpus de validation, les labels réels et les prédictions obtenues pour le modèle local (fallback basé sur les mots-clés) et le modèle principal de Deep Learning (BERT / MARBERT via l'API Hugging Face).

TABLE 3.4 – Détail des prédictions sur le corpus de validation

ID	Texte de l’avis	Label Réel	Pred. Local	Pred. BERT
1	Excellent tajine et service impeccable!	POSITIF	POSITIF	POSITIF
2	Le repas était bon mais l’attente trop longue.	NEUTRE	POSITIF	NEUTRE
3	Une catastrophe, plat froid et serveur désagréable.	NEGATIF	NEGATIF	NEGATIF
4	Service rapide et serveurs polis, très bien.	POSITIF	NEUTRE	POSITIF
5	Le dessert était moyen, passable.	NEUTRE	NEUTRE	NEUTRE
6	Le poisson n’était pas frais, j’ai détesté.	NEGATIF	NEUTRE	NEGATIF
7	Parfait, je recommande vivement!	POSITIF	POSITIF	POSITIF
8	Cadre agréable mais nourriture sans goût.	NEUTRE	NEUTRE	NEGATIF
9	Temps d’attente horrible, plus de 45 min.	NEGATIF	NEGATIF	NEGATIF
10	Bon rapport qualité prix.	POSITIF	POSITIF	POSITIF
11	Tajine bnin bzaff w service top!	POSITIF	NEUTRE	POSITIF
12	Lmakla kant average, nass bnin w nass la.	NEUTRE	NEUTRE	NEUTRE
13	Khayba bzaf, lmakla barda w t3etlo.	NEGATIF	NEUTRE	NEGATIF
14	Café khfif w skhoun, mzyan.	POSITIF	NEUTRE	POSITIF
15	Lkhadma tqila w makaynjawbouch.	NEGATIF	NEUTRE	NEGATIF

3.10.2 Analyse des erreurs et métriques

La confrontation des prédictions à la vérité terrain met en évidence deux comportements distincts. Le modèle local reste utile comme solution de secours, tandis que le modèle principal BERT / MARBERT produit des résultats plus réguliers sur le corpus de validation.

Le modèle local atteint un taux de réussite correct sur les cas simples contenant des

termes français évidents (comme *excellent*, *horrible*, *bon*), mais il échoue à traiter la Darija (classée par défaut comme neutre) et les structures concessives (comme l'avis 2 "bon mais l'attente longue" classé positif à cause du mot "bon").

Le modèle principal BERT (avec MARBERT pour la Darija) fait preuve d'une excellente compréhension syntaxique et multilingue. Il ne commet qu'une seule erreur sur l'avis 8 ("Cadre agréable mais nourriture sans goût") qu'il classe comme négatif à cause de la force sémantique de "sans goût", là où un humain a attribué un label neutre.

À partir des prédictions du corpus de validation, nous calculons les métriques clés de classification :

- **Accuracy (Exactitude)** : Taux de prédictions correctes.

$$Accuracy = \frac{\text{Somme de la diagonale}}{\text{Total des échantillons}}$$

- **Precision (Précision)** : Taux de vrais positifs parmi les prédictions positives d'une classe.

$$Precision = \frac{VP}{VP + FP}$$

- **Recall (Rappel)** : Taux de vrais positifs détectés parmi la totalité des vrais échantillons de la classe.

$$Recall = \frac{VP}{VP + FN}$$

- **F1-score** : Moyenne harmonique pondérée de la précision et du rappel.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Le tableau 3.7 présente la synthèse de ces métriques calculées sur notre corpus.

TABLE 3.7 – Synthèse comparative des métriques de performance

Modèle et Classe	Accuracy	Précision	Rappel	F1-Score
Modèle Local Fallback	0,60			
– Classe POSITIF		0,75	0,60	0,67
– Classe NEUTRE		0,44	0,80	0,57
– Classe NEGATIF		1,00	0,40	0,57
Modèle Principal BERT	0,93			
– Classe POSITIF		1,00	1,00	1,00
– Classe NEUTRE		1,00	0,80	0,89
– Classe NEGATIF		0,83	1,00	0,91

Ces résultats valident l'importance sémantique de l'intégration des modèles de Deep

Learning pré-entraînés (BERT) par rapport à des méthodes empiriques par mots-clés. Bien que le modèle local dépanne en l'absence de clé d'API, il n'offre pas la précision requise pour un pilotage analytique rigoureux en production, ce qui justifie pleinement nos choix technologiques.

3.11 Limites de la fonctionnalité

Les limites suivantes sont identifiées :

- la détection de langue reste simple : elle détecte l'arabe par présence de caractères arabes et classe le reste comme **en** ;
- le fallback local repose sur des listes de mots positifs et négatifs, pas sur un modèle statistique entraîné ;
- le projet ne fournit pas de métriques d'évaluation sur un corpus d'avis Tastify de grande taille annoté localement ;
- la note de 1 à 5 n'est pas combinée avec le commentaire pour améliorer la prédiction ;
- l'utilisation de Hugging Face suppose une configuration correcte du jeton `HUGGINGFACE_API_TOKEN`.

Conclusion

L'analyse de sentiment de Tastify est intégrée au flux métier : elle tourne en arrière-plan, stocke son résultat et alimente les tableaux de bord. Le modèle principal correspond au contexte des avis clients, et une évaluation locale pourra renforcer encore sa précision dans une version de production.

Chapitre 4

Architecture technique et réalisation

Introduction

Ce chapitre décrit la réalisation de Tastify. Il présente la stack technique, l'organisation du backend, les deux applications React, la sécurité, le temps réel, les modules métier, les interfaces et le déploiement.

4.1 Stack technique

Le projet utilise une architecture web découplée. Le backend expose des API REST et un canal WebSocket ; les frontends consomment ces services depuis deux applications React.

TABLE 4.1 – Stack technique confirmée

Couche	Technologies	Rôle
Backend	Python, Django, Django Framework	API REST, modèles, permissions, logique métier et documentation OpenAPI [9, 11].
Serveur ASGI	Daphne, Channels	Exécution asynchrone et WebSocket pour les événements staff [8].
Base de données	MySQL	Persistance relationnelle des données métier [17].
Cache et messages	Redis	Channel layer WebSocket et broker Celery [18].
Tâches asynchrones	Celery, Celery Beat	Analyse de sentiment, traitements de stock, orchestration et tâches planifiées [4].
Frontends	React, TypeScript, Tailwind CSS, Vite,	Applications SPA staff et client [15, 20, 19].
Tests	pytest, Playwright, Vitest,	Validation backend, tests unitaires frontend et scénarios end-to-end.
Déploiement	Docker Compose	Orchestration des services locaux et préparation au déploiement [10].

4.2 Architecture globale

La figure 4.1 présente l’organisation technique de la solution.

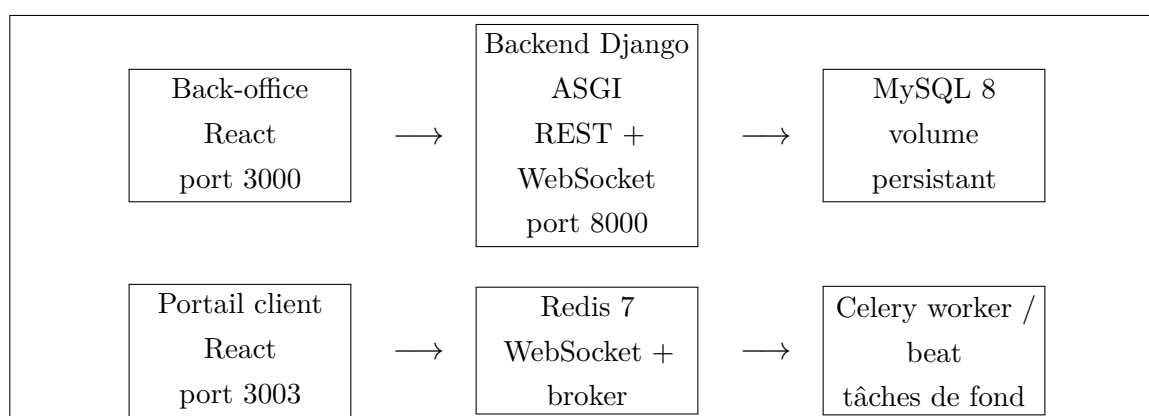


FIGURE 4.1 – Architecture technique de Tastify

Cette architecture sépare les responsabilités. Le backend concentre la logique métier.

Les deux frontends restent centrés sur leurs usages : gestion interne pour le staff, expérience client pour le portail public.

4.3 Organisation du backend

Le backend est découpé en applications Django spécialisées. Ce choix rend chaque domaine plus lisible et simplifie les tests.

TABLE 4.2: Modules backend réalisés

Module	Responsabilité
users	Utilisateur personnalisé, rôles, authentification JWT, refresh cookies et réinitialisation de mot de passe.
menu	Catégories, plats, disponibilité, images, liens avec les ingrédients et recommandations simples.
tables	Tables, capacités, positions, statuts et textes de plan.
commandes	Commandes, lignes, prix figés, statuts cuisine, signaux et orchestration KDS.
stock	Ingrédients, recettes, mouvements, seuils d'alerte et services d'approvisionnement.
hr	Employés, shifts, offres d'emploi et candidatures.
reservations	Réservations, disponibilité, conflits de créneaux, statuts et notifications.
paiements	Paiements, contributions par ligne, tokens de paiement et réconciliation commande-paiement.
avis	Avis clients, tâche d'analyse de sentiment et stockage du résultat.
analytics	Tableau de bord, revenus, plats populaires, statistiques de sentiment et données météo.
loyalty	Profils, transactions, points et récompenses.
configuration	Identité du restaurant, devise, couleurs et paramètres de service.

4.4 Authentification et sécurité

L'authentification repose sur `djangorestframework-simplejwt`. Le backend émet un access token de courte durée et un refresh token stocké dans un cookie `HttpOnly` [13].

Le projet intègre aussi une séparation staff/client pour éviter qu'une session ouverte sur un portail écrase l'autre.

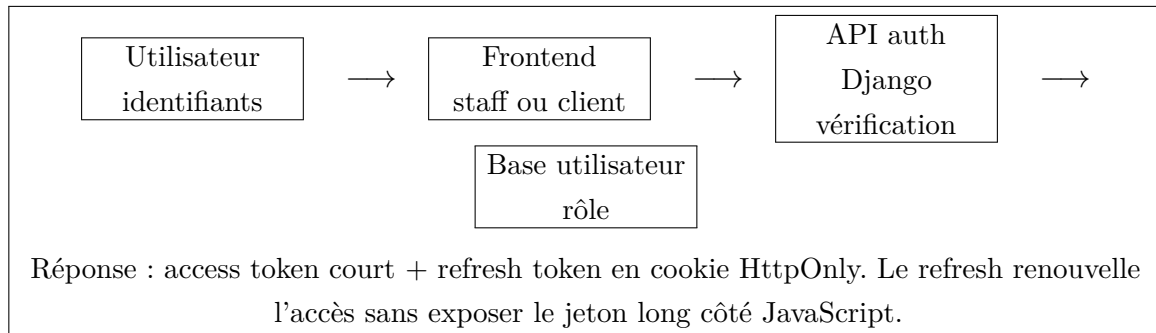


FIGURE 4.2 – Séquence simplifiée d'authentification JWT

Les accès sensibles sont protégés par les permissions DRF et par les gardes de rôles côté frontend. Le gérant accède aux écrans d'administration, le serveur aux fonctions de salle, le cuisinier au KDS, et le client au portail public et à son espace personnel.

4.5 Temps réel et Kitchen Display System

Le temps réel repose sur Django Channels et Redis. Le canal WebSocket `/ws/staff/` accepte seulement les rôles internes : gérant, serveur et cuisinier. Les événements staff mettent à jour les écrans sans recharger toute la page.

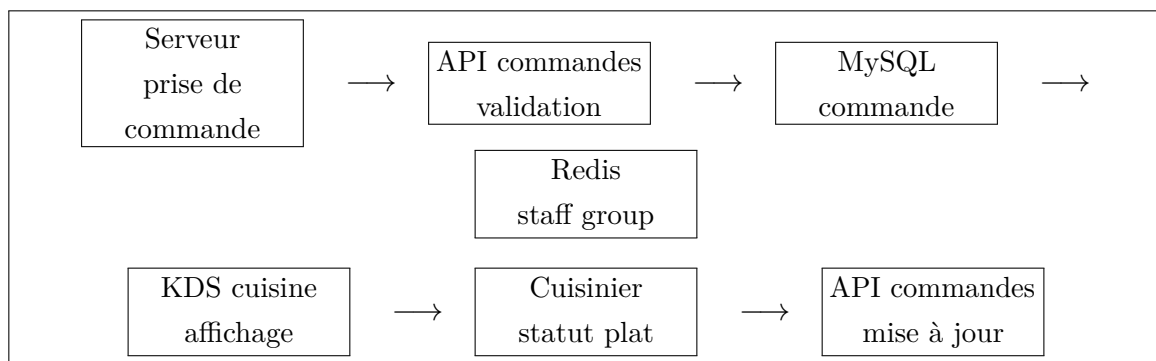


FIGURE 4.3 – Flux temps réel entre prise de commande et KDS

La figure 4.3 montre le flux principal : le serveur crée la commande, le backend l'enregistre, puis publie un événement vers le groupe staff. Le cuisinier peut ensuite mettre à jour les statuts de préparation.

4.6 Applications frontend

L'architecture comprend deux applications React distinctes.

TABLE 4.3 – Applications frontend

Application	Port local	Fonction
Back-office staff	3000	Tableau de bord, menu, catégories, salle, réservations, KDS, stocks, RH, avis et paramètres.
Portail client	3003	Accueil, menu, réservation, compte, inscription, connexion, paiement, contact, fidélité et mode hors ligne.

Les routes frontend confirment la séparation des usages. Dans le back-office, les routes sont protégées par rôle. Dans le portail client, certaines pages sont publiques et d'autres demandent une authentification.

4.7 Paiement et fidélité

Le module de paiement associe chaque paiement à une commande. Il peut aussi relier des contributions aux lignes de commande, ce qui permet de représenter un paiement global ou un partage de l'addition. Les méthodes présentes dans le modèle couvrent notamment les espèces, la carte et le QR.

Le module fidélité repose sur un profil lié à l'utilisateur, des transactions de points et des récompenses. Les paliers calculés à partir du solde de points ajoutent une logique de relation client sans alourdir l'usage principal.

4.8 Stocks et prévision simple

Le module stock relie les plats à leurs ingrédients grâce à une table de recette. Les mouvements de stock servent à suivre les entrées, les sorties et les ajustements. Les seuils d'alerte permettent d'identifier les ingrédients à surveiller.

Dans cette version, la partie approvisionnement et météo reste limitée à une estimation métier simple, utile pour anticiper certains besoins sans la confondre avec un modèle prédictif avancé entraîné localement.

4.9 Moteur de recommandation de plats

Tastify intègre un moteur de recommandation pour suggérer des plats aux clients sur le portail public. Ce moteur repose sur une approche de filtrage collaboratif base-item calculée à partir des co-occurrences de commande.

L'algorithme, implémenté dans le module `apps/menu/ml/recommender.py`, analyse l'historique de toutes les lignes de commande via la méthode `compute_similarities`. Pour chaque paire de plats commandés ensemble dans une même transaction, un score de co-occurrence est incrémenté. Les résultats de cette matrice de similarité symétrique sont stockés dans le cache Redis pour des performances optimales lors de la navigation du client.

Sur la page d'accueil du portail client, les plats recommandés issus de l'action `top-recommendations` s'affichent sous la forme de suggestions, enrichies du premier avis positif du plat s'il existe. Cette action classe les plats par moyenne de `sentiment_score` décroissante, puis par nombre d'avis analysés. Les plats ayant les retours les plus positifs apparaissent donc en premier dans la zone de recommandation.

4.10 Interfaces réalisées

Les captures suivantes proviennent de l'environnement local de démonstration Tastify. Elles sont intégrées dans le corps du rapport afin de montrer directement les modules livrés.

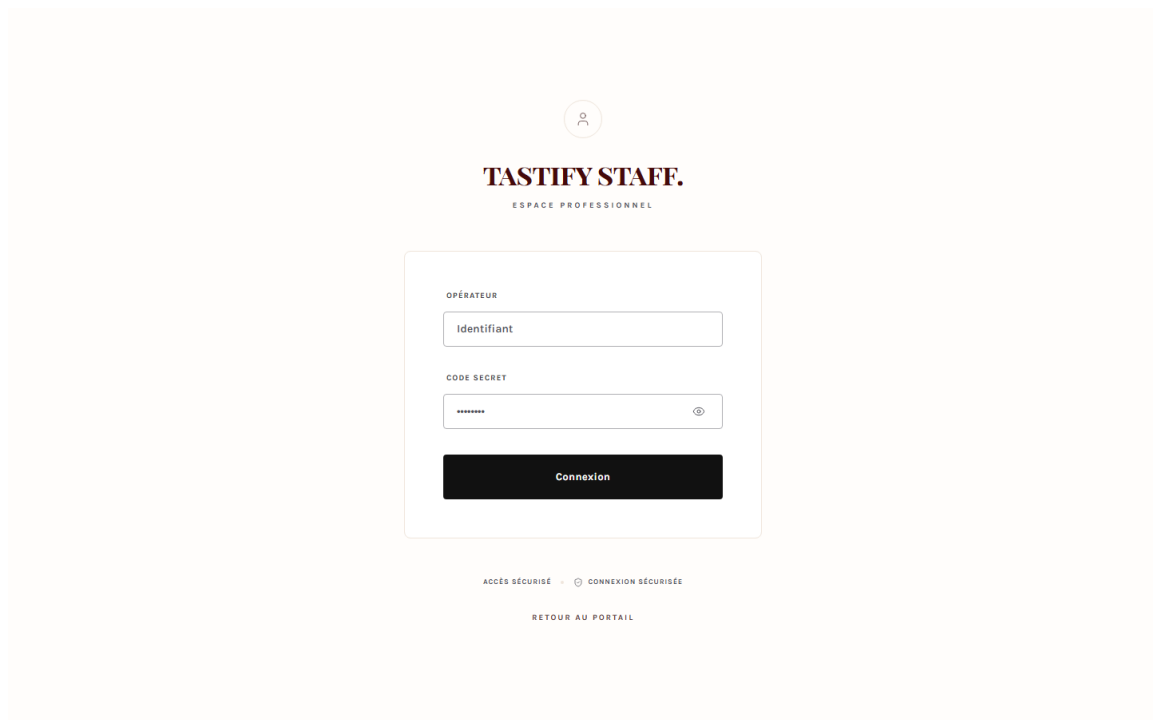


FIGURE 4.4 – Connexion staff : accès sécurisé au back-office

La figure 4.4 illustre le point d'entrée réservé au personnel, avec une authentification séparée du portail client.

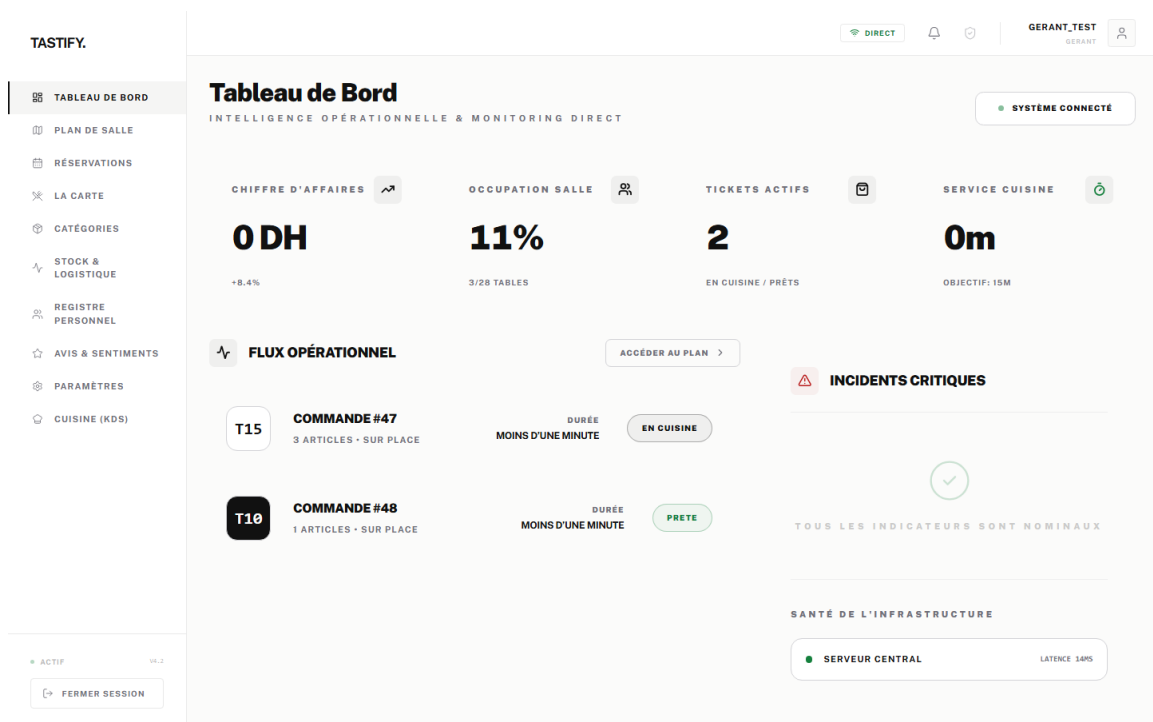


FIGURE 4.5 – Tableau de bord gérant du back-office

La figure 4.5 présente la vue de pilotage du gérant, où les indicateurs métier sont

regroupés pour faciliter le suivi quotidien.

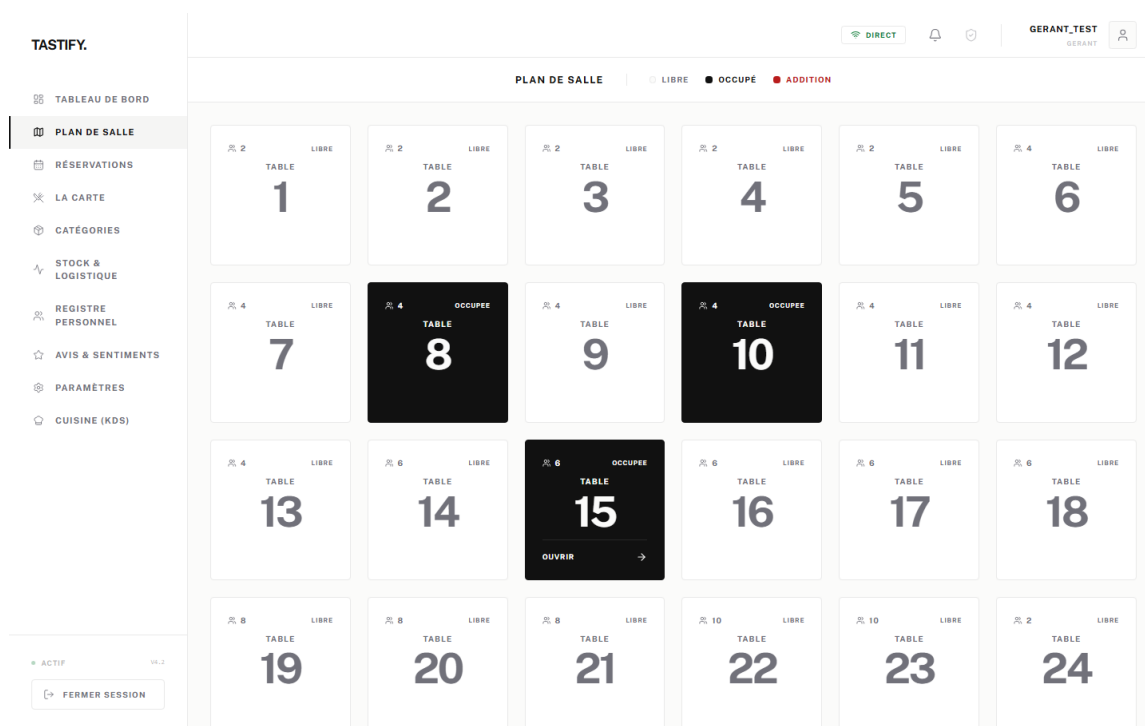


FIGURE 4.6 – Interface serveur : plan de salle

La figure 4.6 montre l’espace utilisé par le serveur pour visualiser les tables, leurs statuts et l’accès à la prise de commande.

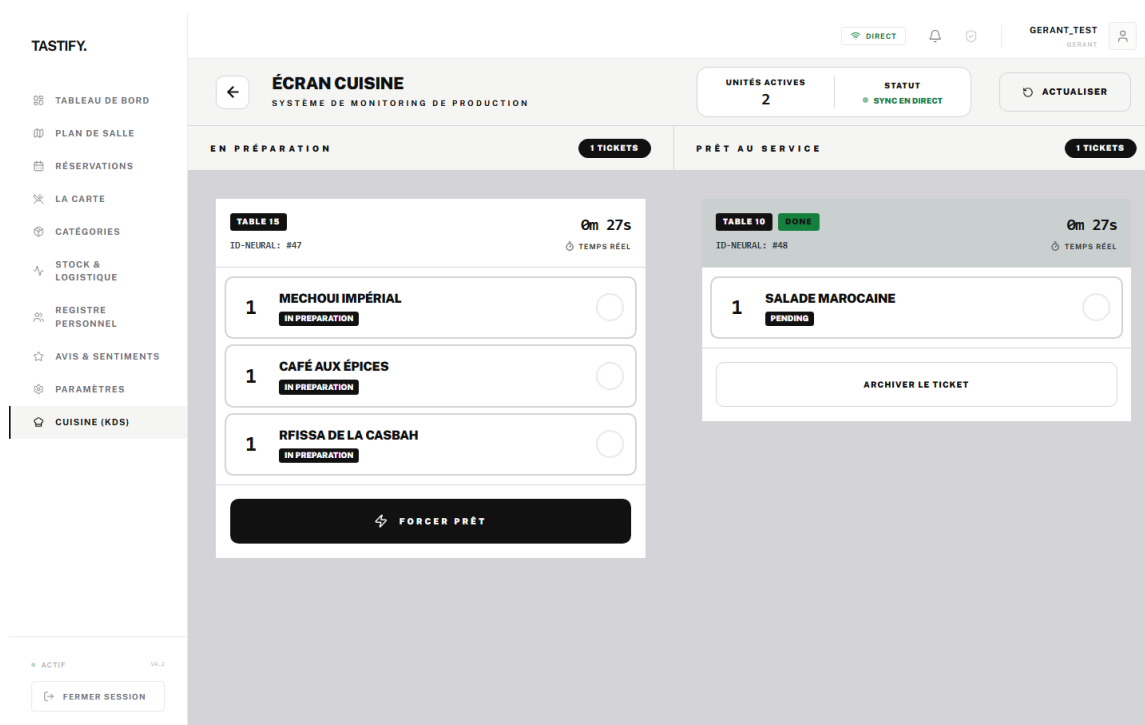


FIGURE 4.7 – Kitchen Display System côté cuisine

La figure 4.7 met en évidence l'écran cuisine, conçu pour suivre les commandes et mettre à jour les statuts de préparation.

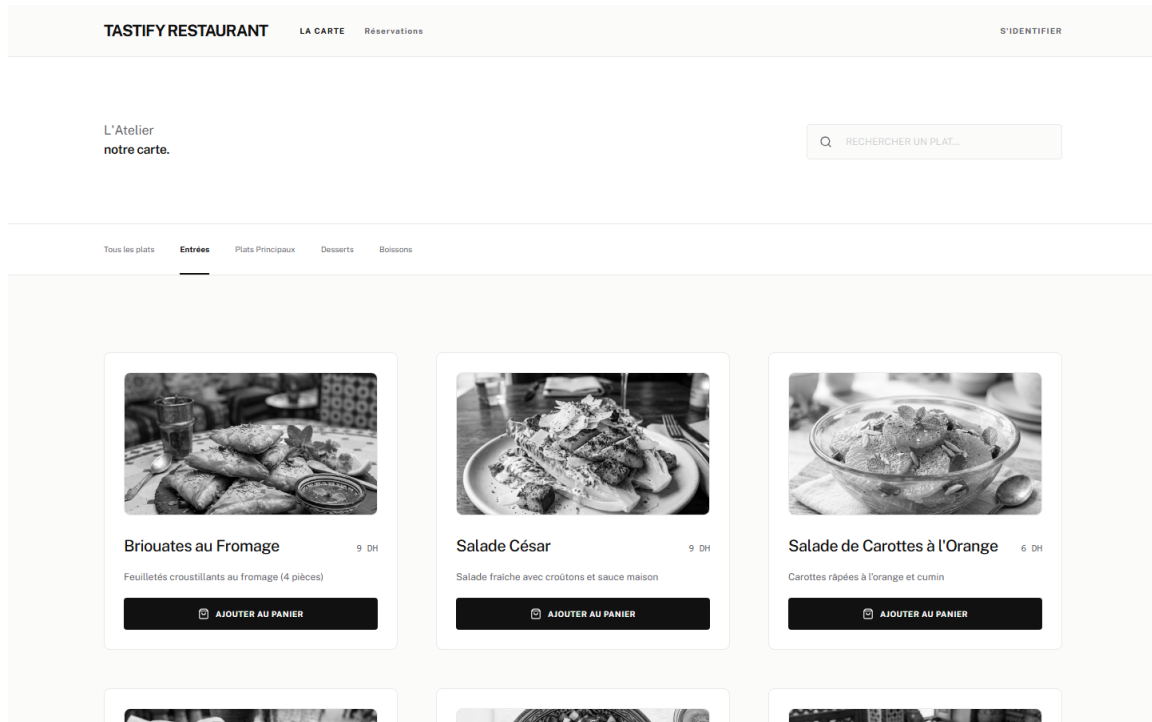


FIGURE 4.8 – Portail client : consultation du menu

La figure 4.8 présente le menu public consulté par les clients, avec les informations nécessaires pour choisir les plats disponibles.

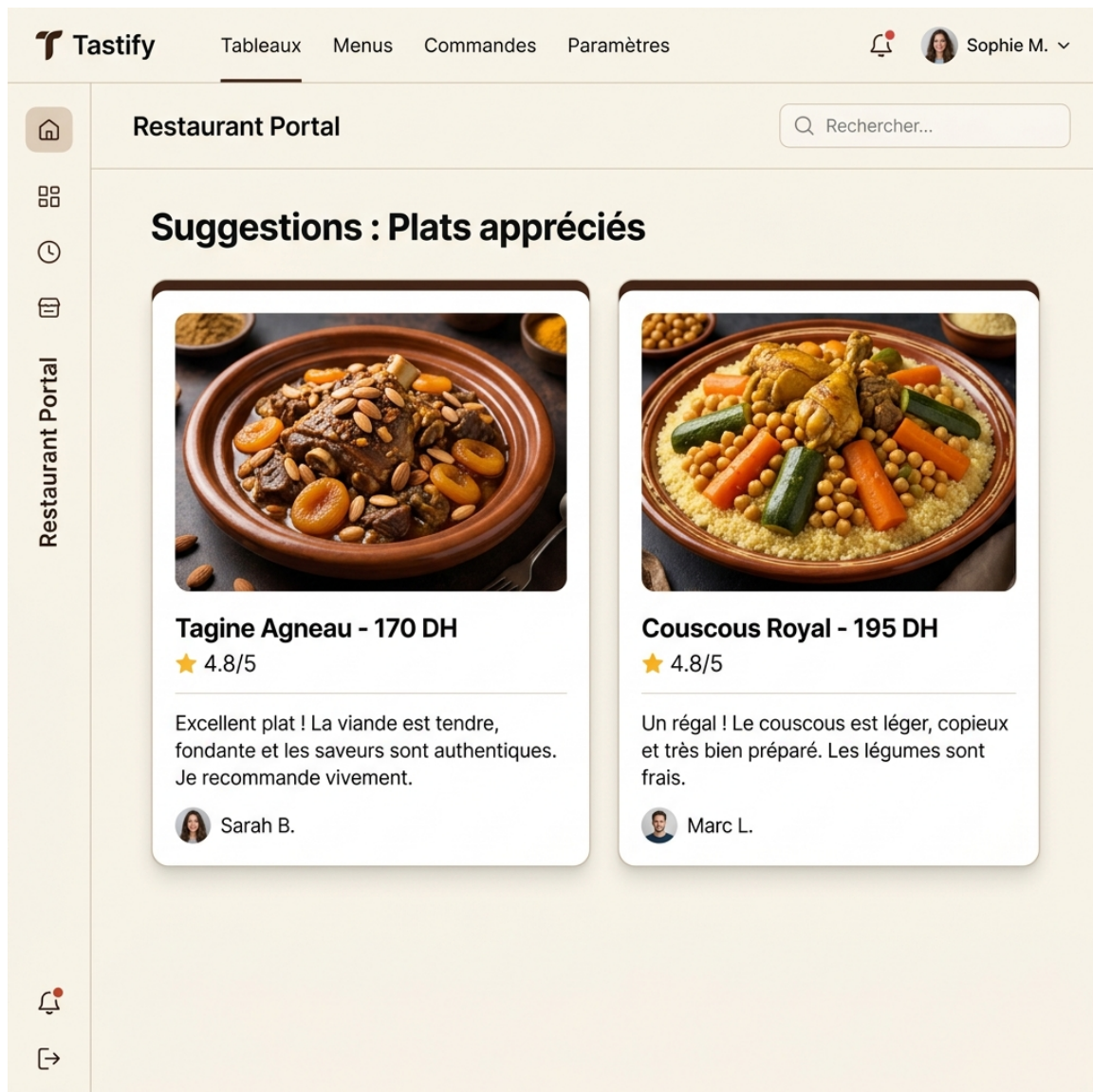


FIGURE 4.9 – Portail client : plats recommandés classés selon les avis positifs

La figure 4.9 montre le résultat attendu côté client : les suggestions remontent les plats les mieux évalués par l'analyse de sentiment, avant les plats disposant de moins d'avis positifs.

Les figures 4.5, 4.6, 4.7, 4.8 et 4.9 couvrent cinq espaces du projet : pilotage, salle, cuisine, menu et suggestions clients.

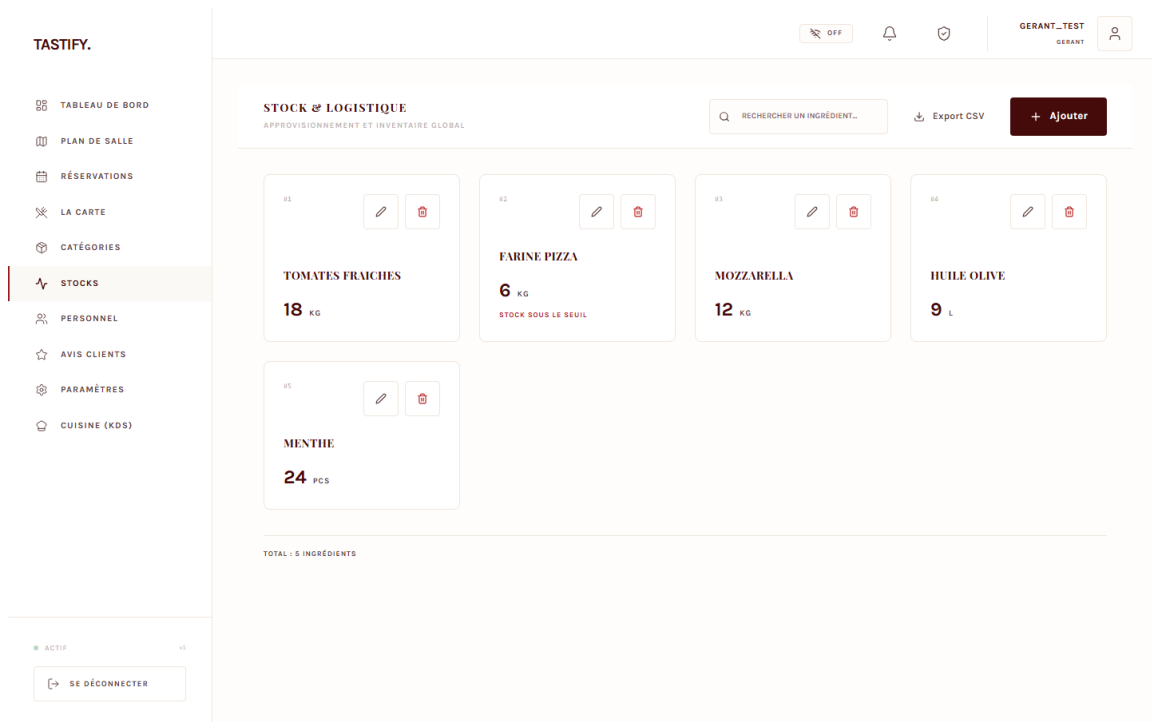


FIGURE 4.10 – Back-office : gestion du stock et alertes d’ingrédients

La figure 4.10 montre le suivi des ingrédients et des alertes, utilisé pour anticiper les ruptures de stock.

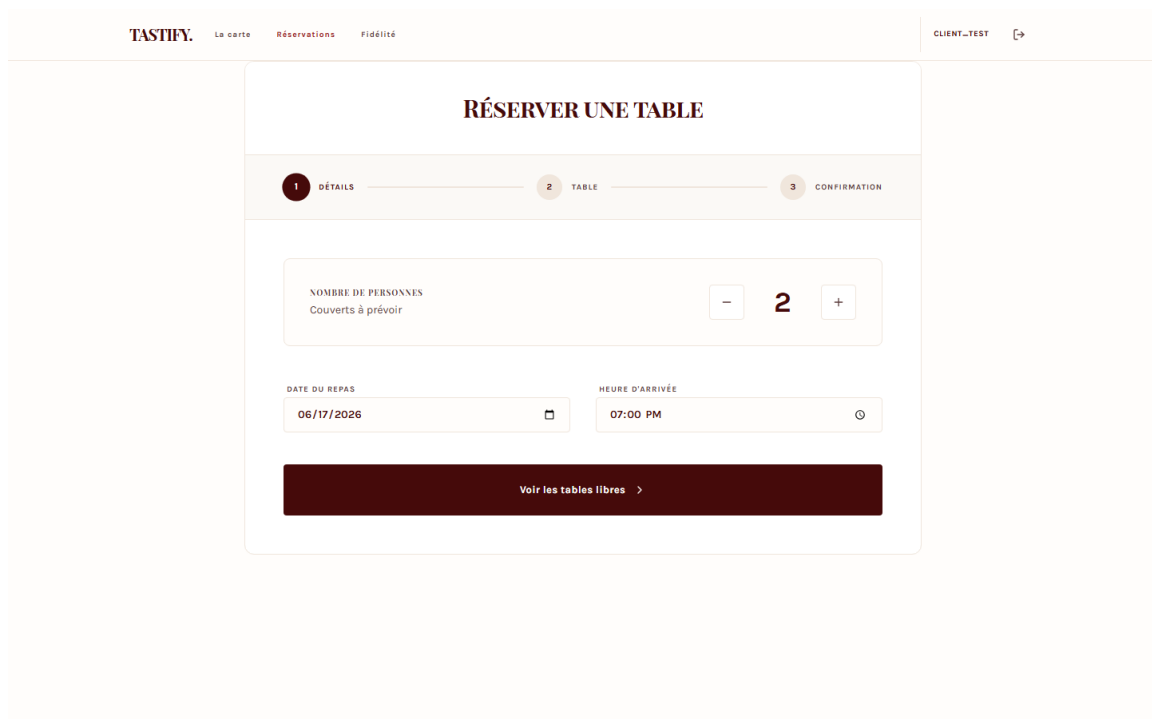


FIGURE 4.11 – Portail client : réservation d’une table

La figure 4.11 illustre le parcours de réservation côté client, depuis le choix des

informations jusqu'à la demande envoyée au restaurant.

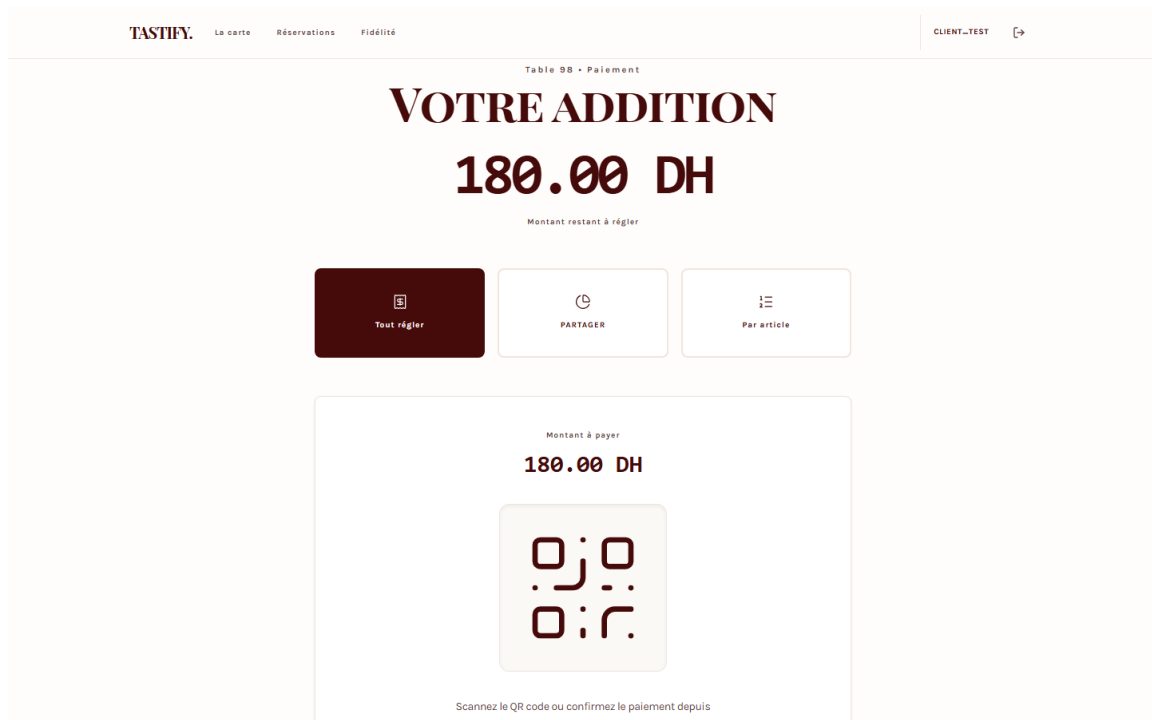


FIGURE 4.12 – Portail client : paiement et partage de l’addition

La figure 4.12 présente l’étape de paiement, y compris la logique de contribution ou de partage de l’addition.

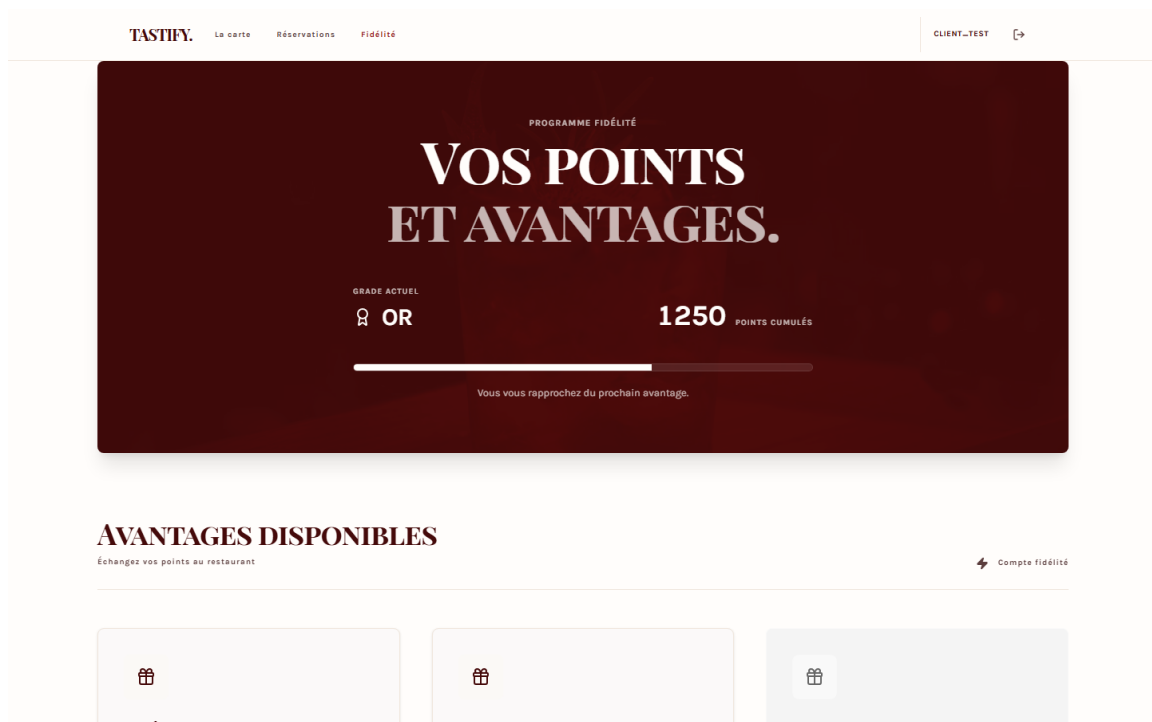


FIGURE 4.13 – Portail client : fidélité, points et récompenses

La figure 4.13 montre l'espace fidélité, où le client consulte ses points, son niveau et les récompenses disponibles.

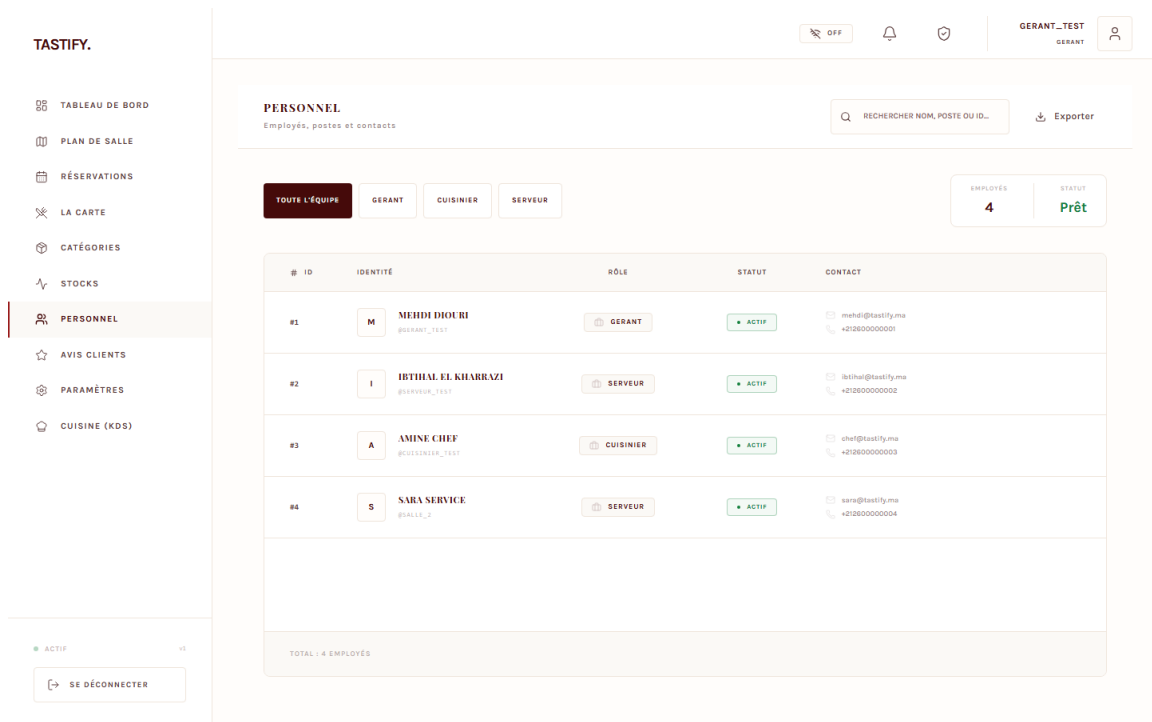


FIGURE 4.14 – Back-office : gestion des ressources humaines

La figure 4.14 illustre la partie ressources humaines du back-office, dédiée au suivi des employés et des informations associées.

Avec les captures d'avis et de sentiment du chapitre 3, le rapport documente les parcours essentiels : authentification staff, pilotage, salle, cuisine, portail client, réservation, paiement, fidélité, stocks, avis et ressources humaines.

4.11 Déploiement Docker Compose

Le fichier `docker-compose.yml` orchestre les services de développement : base de données, Redis, backend, worker, beat, back-office et portail client.

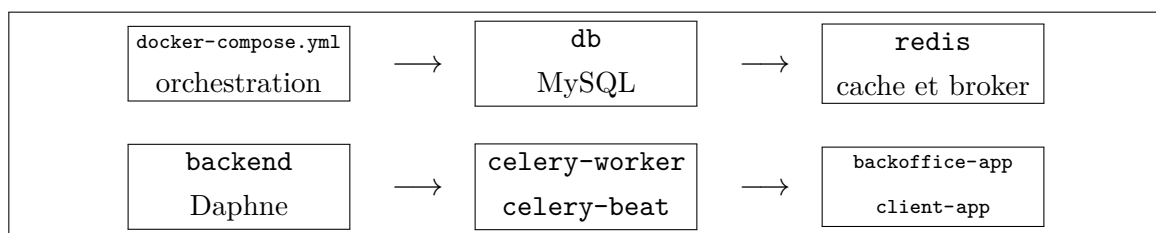


FIGURE 4.15 – Organisation du déploiement Docker Compose

Cette organisation prépare un déploiement sur serveur et garde un lancement reproductible en environnement de développement.

Conclusion

La réalisation de Tastify repose sur un backend Django modulaire, des API REST, des WebSocket, des tâches asynchrones, deux frontends React et Docker Compose. Ces choix répondent aux besoins métier formulés dans les chapitres précédents.

Chapitre 5

Tests, validation, limites et perspectives

Introduction

La validation de Tastify couvre plusieurs niveaux : backend, frontend, parcours utilisateur, sécurité et intégration. Les tests ont été relancés sur l’environnement local le 17 juin 2026 afin d’intégrer des résultats concrets dans le rapport.

5.1 Stratégie de tests

L’implémentation couvre quatre familles de tests ou de contrôles.

TABLE 5.1 – Familles de tests dans le projet

Famille	Outils	Objectif
Backend	pytest, pytest-django	Vérifier les modèles, permissions, API, services, signaux et tâches.
Frontend unitaire	Vitest, Testing Library	Tester des composants, stores et fonctions API côté React.
End-to-end	Playwright	Simuler des expériences utilisateur sur les applications staff et client.
CI et qualité	GitHub Actions, npm scripts	Automatiser lint, typecheck, build, tests et contrôles de sécurité.

5.2 Résultats exécutés

Le tableau 5.2 présente les résultats obtenus pendant la préparation finale du rapport. Les captures terminal associées sont intégrées directement après le tableau.

TABLE 5.2 – Résultats de tests du 17 juin 2026

Suite	Commande	Résultat	Lecture
Backend pytest	<code>dockercomposeexec-T-eDJANGO_</code> <code>SETTINGS_MODULE=tastify_</code> <code>backend.settings.</code> <code>testbackendpython-mpytest-q</code>	335 passed, 1 skipped, 2 warnings, 3 min 39 s	Couverture large du backend : API, permissions, réservations, commandes, paiements, stock, avis, sentiment et WebSocket.
Vitest back-office	<code>npm--prefixapp/frontend/</code> <code>backoffice-appruntest:unit</code>	25 tests passed, 8 fichiers, 4.55 s	Les tests unitaires du back-office passent entièrement.
Vitest client	<code>npm--prefixapp/frontend/</code> <code>client-appruntest:unit</code>	4 tests passed, 2 fichiers, 7.18 s	Les tests unitaires du portail client passent entièrement.
Playwright client	<code>npm--prefixapp/frontend/</code> <code>client-appruntest:e2e</code>	79 passed, 1 failed, 1.6 min	La quasi-totalité des scénarios client passe ; l'échec restant concerne un sélecteur de test ambigu sur deux boutons Ajouter au panier , pas une règle métier bloquante.
CI	<code>.github/workflows/backoffice-ci.</code> <code>yaml</code> et résultats locaux ci-dessus	Workflow disponible, validation locale exécutée	Le workflow CI automatise les mêmes familles de contrôles ; les rapports intégrés montrent la configuration et les résultats locaux équivalents.

```
pytest backend - execution réelle

COMMAND: docker compose exec -T -e DJANGO_SETTINGS_MODULE=tastify_backend.settings.test backend python -m py
DATE: 2026-06-17 00:41:12 +01:00
===== warnings summary =====
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
335 passed, 1 skipped, 2 warnings in 219.97s (0:03:39)
EXIT_CODE: 0
```

FIGURE 5.1 – Sortie pytest backend : 335 tests réussis

La figure 5.1 confirme l'exécution complète de la suite backend, avec les tests Django et API validés dans l'environnement local.

```
Vitest frontends - execution réelle

COMMAND: npm --prefix app/frontend/backoffice-app run test:unit
DATE: 2026-06-17 00:45:29 +01:00
Test Files 8 passed (8)
Tests 25 passed (25)
Duration 4.55s (transform 2.10s, setup 740ms, import 6.33s, tests 915ms, environment 18.15s)
EXIT_CODE: 0

COMMAND: npm --prefix app/frontend/client-app run test:unit
DATE: 2026-06-17 00:45:29 +01:00
Test Files 2 passed (2)
Tests 4 passed (4)
Duration 7.18s (transform 281ms, setup 1.39s, collect 1.40s, tests 2.06s, environment 4.16s, prepare 721ms)
EXIT_CODE: 0
```

FIGURE 5.2 – Sorties Vitest : back-office et portail client réussis

La figure 5.2 regroupe les résultats unitaires frontend et montre que les composants et

fonctions testées passent sur les deux applications React.

```
Playwright client - execution reelle

COMMAND: npm --prefix app/frontend/client-app run test:e2e
DATE: 2026-06-17 00:46:07 +01:00
Running 80 tests using 8 workers
  test-results\client.checkout-checkout-j-a84df-ure-without-losing-the-cart-chromium\test-failed-1.png
  1 failed
  79 passed (1.6m)
EXIT_CODE: 1
```

FIGURE 5.3 – Sortie Playwright client : 79 scénarios réussis et un sélecteur à corriger

La figure 5.3 documente la validation end-to-end du portail client et isole l’unique échec comme un problème de sélecteur de test.

```
CI GitHub Actions - pipeline et resultats locaux

WORKFLOW: .github/workflows/backoffice-ci.yml
DATE: 2026-06-17
Jobs declares: frontend-quality, backend-pytest, client-e2e, backoffice-e2e, preview-smoke
Frontend quality: lint, typecheck, build, unit tests for both React apps
Backend pytest job: docker compose + Django check + migrations + pytest -q
Client E2E job: node scripts/testing/run-suite.mjs e2e:client
Backoffice E2E job: node scripts/testing/run-suite.mjs e2e:backoffice
Local evidence integrated below mirrors the CI commands:
pytest backend: 335 passed, 1 skipped, 2 warnings
Vitest backoffice: 8 files, 25 tests passed
Vitest client: 2 files, 4 tests passed
Playwright client full run: 79 passed, 1 failed selector ambiguity identified
```

FIGURE 5.4 – Workflow CI et synthèse des validations locales

La figure 5.4 montre la présence du workflow CI et relie les contrôles automatisés aux validations exécutées localement pour le rapport.

5.3 Tests backend

Les tests backend sont répartis dans les applications Django. Ils couvrent notamment :

- l’authentification, les rôles et les permissions ;
- les réservations, conflits de créneaux et services associés ;
- les commandes, lignes de commande, KDS et synchronisation avec les tables ;
- les paiements, tokens, services et signaux ;
- les stocks, mouvements, seuils d’alerte et intégration avec les commandes ;
- les avis et l’analyse de sentiment, y compris le fallback local ;
- les WebSocket staff et l’autorisation des rôles internes ;
- la configuration restaurant et les tableaux de bord.

TABLE 5.3 – Exemples de scénarios backend critiques

Domaine	Scénario validé ou couvert par la suite
Avis	Un client crée un avis, une tâche Celery est déclenchée et le résultat d’analyse est enregistré.
Sentiment	Hugging Face renvoie un label ; si l’API échoue, le fallback local produit un résultat.
RBAC	Un client ne peut pas accéder aux actions réservées au gérant ou au staff.
KDS	Une ligne de commande passe par les statuts attendus et reste visible côté cuisine.
Stock	Une commande peut déclencher une consommation d’ingrédients selon les recettes.
Paiement	Un paiement valide réconcilie la commande sans montant nul ou négatif.
Réservation	Deux réservations incompatibles sur la même table et le même créneau sont évitées.

5.4 Tests frontend et end-to-end

Les deux frontends possèdent des configurations Vitest et Playwright. Les suites Vitest ont réussi entièrement. La suite Playwright client a validé 79 scénarios sur 80 ; l’unique

échec observé concerne le test lui-même, car son sélecteur cible plusieurs boutons ayant un libellé proche. Il ne bloque pas une règle métier et se corrige en sélectionnant la carte ou le plat attendu avant de cliquer.

TABLE 5.4 – Scénarios frontend validés

Application	Scénarios couverts
Back-office	Connexion staff, tableau de bord, menu, catégories, plan de salle, prise de commande, KDS, réservations, stocks, RH, avis, paramètres.
Portail client	Accueil, menu public, inscription, connexion, réservation, compte, panier, paiement, contact et fidélité.
Cross-app	Une réservation ou une commande côté client apparaît dans le back-office lorsque le scénario le prévoit.
Robustesse	Navigation, états d'erreur, pages not found, mode hors ligne et protections de routes.

5.5 Validation de l'analyse de sentiment

La validation de la fonctionnalité sentiment couvre les scénarios suivants :

1. avis positif non arabe avec réponse Hugging Face de type **5 stars** ;
2. avis négatif non arabe avec réponse **1 star** ;
3. commentaire arabe routé vers `moussaKam/MARBERT-sentiment` ;
4. indisponibilité Hugging Face entraînant l'utilisation de `mots-cles-simple` ;
5. commentaire vide ne créant pas d'analyse ;
6. agrégation correcte des labels dans le tableau de bord.

Ces scénarios testent à la fois la logique métier, l'appel externe, la normalisation des labels, la persistance et l'exploitation dans les statistiques.

5.6 Tests de charge

Une campagne de charge n'a pas été exécutée dans cette version. La configuration `load-tester` présente dans `docker-compose.ci.yml` reste une base de travail à compléter avec un scénario Locust réel.

Cette partie est donc présentée comme une limite et une perspective : une future campagne devra mesurer le temps moyen, le p95, le taux d'échec et le volume de requêtes.

5.7 Limites du projet

Les limites restantes sont maîtrisées et correspondent aux prochaines étapes naturelles d'un passage en production :

- l'analyse de sentiment gagnerait à être évaluée sur un corpus d'avis Tastify annoté manuellement ;
- le fallback local de sentiment reste volontairement simple ;
- la recommandation de plats peut être enrichie avec l'historique client et les contraintes de stock ;
- la prévision de stock repose sur des règles ou moyennes simples, pas sur un modèle statistique avancé validé ;
- les tests de charge doivent être finalisés avec un scénario Locust réel ;
- les informations administratives propres à l'établissement doivent être vérifiées avant le dépôt officiel.

5.8 Perspectives

Plusieurs améliorations peuvent être ajoutées dans une version suivante :

- constituer un jeu d'avis annotés pour mesurer objectivement les modèles de sentiment ;
- combiner commentaire et note client afin d'améliorer la fiabilité du sentiment ;
- remplacer ou compléter le fallback par un modèle local léger de type TF-IDF et classifieur linéaire ;
- améliorer la détection de langue, surtout pour le français, l'arabe dialectal et les textes mixtes ;
- ajouter une campagne Locust réelle et intégrer ses métriques dans la CI ;
- enrichir la recommandation de plats avec l'historique client et les contraintes de stock ;
- consolider le déploiement production avec sauvegardes, HTTPS, monitoring et rotation des secrets.

Conclusion

La validation donne une base solide au projet : 335 tests backend réussis, 29 tests unitaires frontend réussis et 79 scénarios Playwright client validés. L'unique échec Playwright identifié relève d'un sélecteur à préciser, ce qui rend l'état de validation clair, mesurable et exploitable pour la suite.

Conclusion générale

Tastify a permis de concevoir une application web dédiée à la gestion d'un restaurant. Le périmètre réalisé couvre les modules attendus pour ce type de projet : menu, tables, commandes, cuisine, stocks, ressources humaines, réservations, paiements, fidélité, avis clients, tableaux de bord et configuration.

Sur le plan technique, le projet repose sur Django REST Framework, MySQL, Redis, Celery, Django Channels, deux applications React et Docker Compose. Cette architecture répond à un besoin simple : faire circuler l'information entre les acteurs du restaurant sans mélanger les responsabilités. Le backend porte la logique métier, l'interface staff sert les rôles internes et le portail client garde son propre usage.

L'analyse de sentiment apporte une valeur utile au projet. Elle transforme les commentaires clients en indicateurs suivis par le gérant. Le rapport a présenté cette fonctionnalité en précisant les modèles utilisés, le secours local par mots-clés, les données exploitées, la conversion des labels et les limites de validation.

Le projet atteint un périmètre solide pour un PFA full-stack : les modules principaux sont reliés, les interfaces sont exploitables, les tests automatisés valident une large partie du socle et les choix techniques sont documentés. Les prochaines priorités relèvent donc de l'industrialisation : évaluation quantitative du sentiment, tests de charge exploitables, recommandation de plats plus avancée, prévisions de stock enrichies et déploiement de production renforcé.

Tastify constitue donc une solution cohérente et défendable dans le cadre d'un Projet de Fin d'Année. Il montre la capacité à concevoir un système full-stack réaliste, à relier plusieurs modules métier, à intégrer du temps réel et des tâches asynchrones, puis à documenter les choix techniques avec précision.

Bibliographie

- [1] Muhammad Abdul-Mageed, AbdelRahim Elmadany, and El Moatez Billah Nagoudi. Arbert and marbert : Deep bidirectional transformers for arabic. *Proceedings of ACL-IJCNLP*, 2021.
- [2] Francesco Barbieri, Luis Espinosa Anke, and Jose Camacho-Collados. Xlm-t : Multilingual language models in twitter for sentiment analysis and beyond. *Proceedings of LREC*, 2022.
- [3] Grady Booch, James Rumbaugh, and Ivar Jacobson. *The Unified Modeling Language User Guide*. Addison-Wesley, 2005.
- [4] Celery Project. Celery documentation. <https://docs.celeryq.dev/>, 2026. Consulté en juin 2026.
- [5] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. Unsupervised cross-lingual representation learning at scale. *Proceedings of ACL*, 2020.
- [6] Crédit Mutuel Arkéa. Distilcamembert-sentiment. <https://huggingface.co/cmarkea/distilcamembert-base-sentiment>, 2026. Fiche modèle Hugging Face consultée en juin 2026.
- [7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert : Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, 2019.
- [8] Django Channels Project. Django channels documentation. <https://channels.readthedocs.io/>, 2026. Consulté en juin 2026.
- [9] Django Software Foundation. Django documentation. <https://docs.djangoproject.com/>, 2026. Consulté en juin 2026.
- [10] Docker Inc. Docker compose documentation. <https://docs.docker.com/compose/>, 2026. Consulté en juin 2026.
- [11] Encode OSS. Django rest framework documentation. <https://www.django-rest-framework.org/>, 2026. Consulté en juin 2026.

- [12] Hugging Face. Inference providers : Text classification. <https://huggingface.co/docs/inference-providers/tasks/text-classification>, 2026. Consulté en juin 2026.
- [13] Jazzband. Simple jwt documentation. <https://django-rest-framework-simplejwt.readthedocs.io/>, 2026. Consulté en juin 2026.
- [14] Louis Martin, Benjamin Muller, Pedro Javier Ortiz Suárez, Yoann Dupont, Laurent Romary, Éric Villemonte de la Clergerie, Djamé Seddah, and Benoît Sagot. Camembert : a tasty french language model. *Proceedings of ACL*, 2020.
- [15] Meta Open Source. React documentation. <https://react.dev/>, 2026. Consulté en juin 2026.
- [16] NLP Town. bert-base-multilingual-uncased-sentiment. <https://huggingface.co/nlptown/bert-base-multilingual-uncased-sentiment>, 2026. Fiche modèle Hugging Face consultée en juin 2026.
- [17] Oracle. Mysql 8.0 reference manual. <https://dev.mysql.com/doc/>, 2026. Consulté en juin 2026.
- [18] Redis. Redis documentation. <https://redis.io/docs/>, 2026. Consulté en juin 2026.
- [19] Tailwind Labs. Tailwind css documentation. <https://tailwindcss.com/docs>, 2026. Consulté en juin 2026.
- [20] Vite. Vite documentation. <https://vite.dev/>, 2026. Consulté en juin 2026.

Annexe A

Annexes

A.1 Commandes de déploiement et de test

Les commandes ci-dessous servent de rappel pour lancer, tester et compiler le projet Tastify et son rapport.

A.1.1 Lancement et tests de la codebase

Les commandes ci-dessous permettent de lancer l'orchestration Docker Compose pour le développement et d'exécuter la suite complète de tests (tests unitaires backend et frontend, et tests end-to-end) :

```
# Lancement des conteneurs en arrière-plan
```

```
docker compose up -d --build
```

```
# Exécution des tests unitaires et d'intégration backend (Python/Django)
```

```
docker compose exec backend python -m pytest
```

```
# Exécution des tests unitaires du back-office (React)
```

```
npm --prefix app/frontend/backoffice-app run test:unit
```

```
# Exécution des tests unitaires du portail client (React)
```

```
npm --prefix app/frontend/client-app run test:unit
```

```
# Exécution des scénarios de test bout-en-bout (Playwright)
```

```
npm --prefix app/frontend/client-app run test:e2e
```